

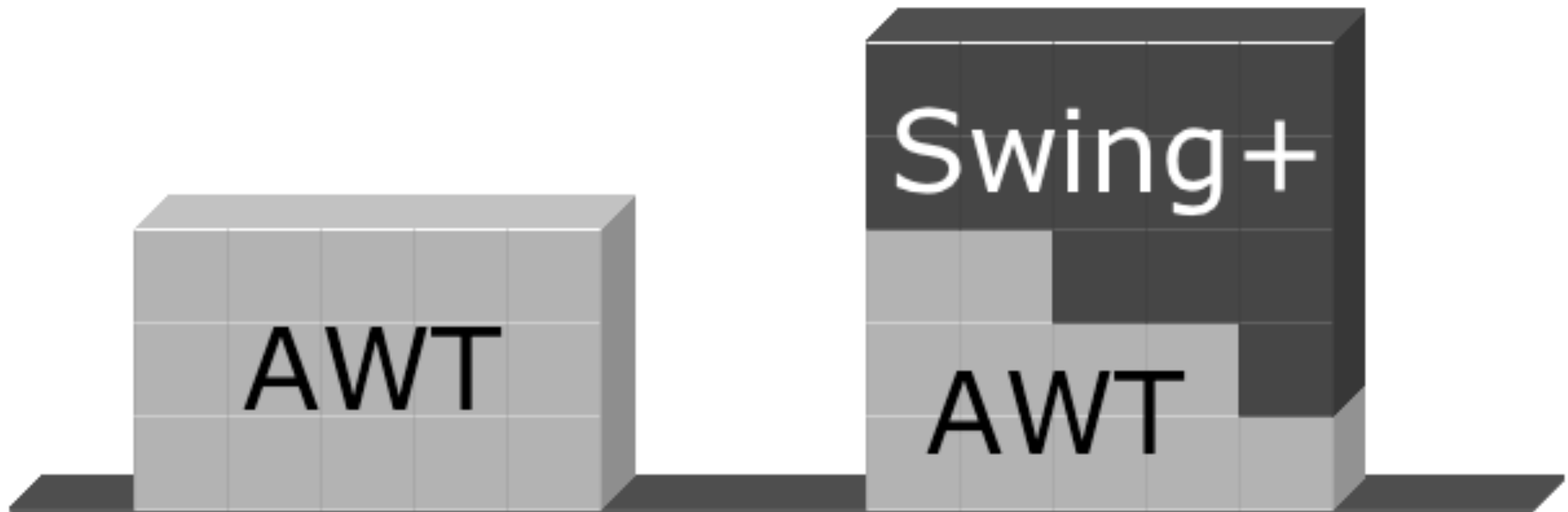
Java Application Design

GUI

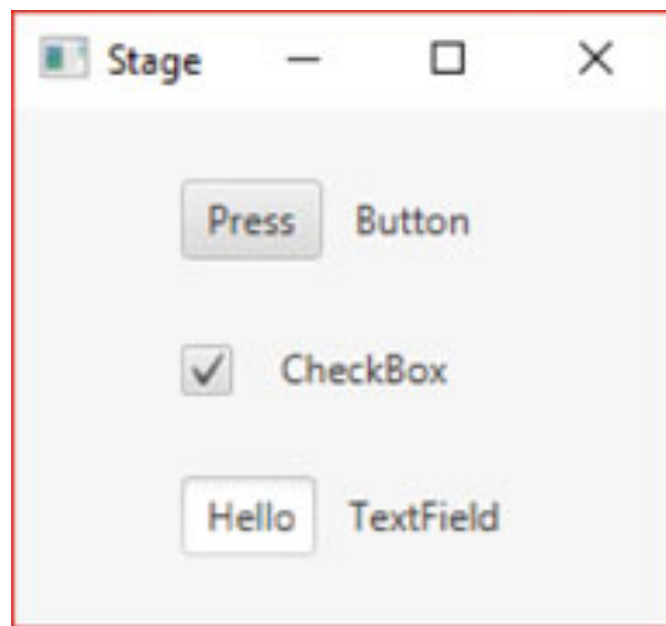
Weng Kai

A Little Story

AWT and Swing



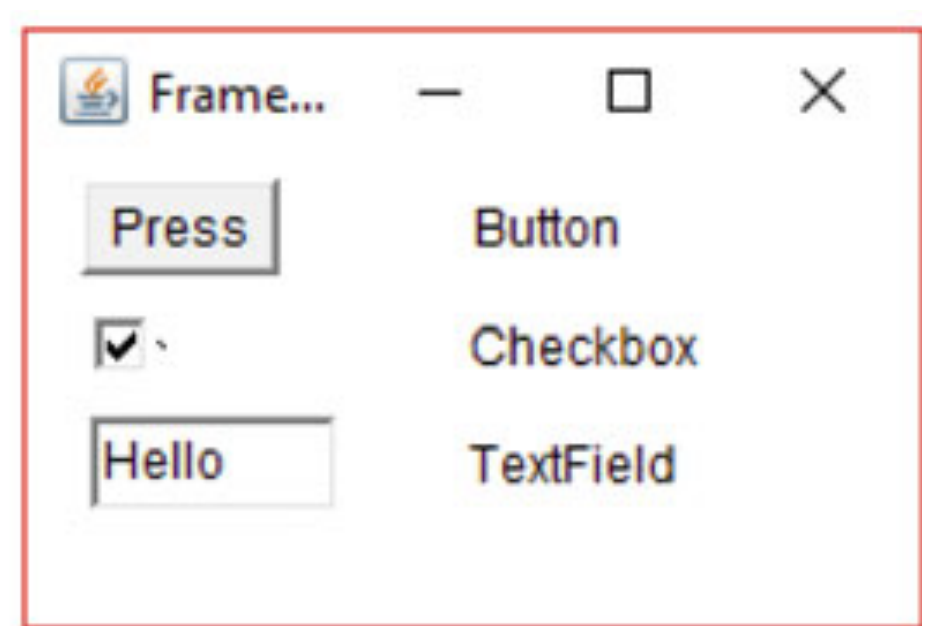
Java FX



(a) JavaFX



(b) Swing



(c) AWT (Windows)

- With the release of Java 8 in 2014 came the latest version of a new technology known as JavaFX.
- And from Java 11, JavaFX has been removed from JDK.....

About the JFC and Swing

- JFC is short for Java Foundation Classes, which encompass a group of features for building graphical user interfaces (GUIs) and adding rich graphics functionality and interactivity to Java applications.



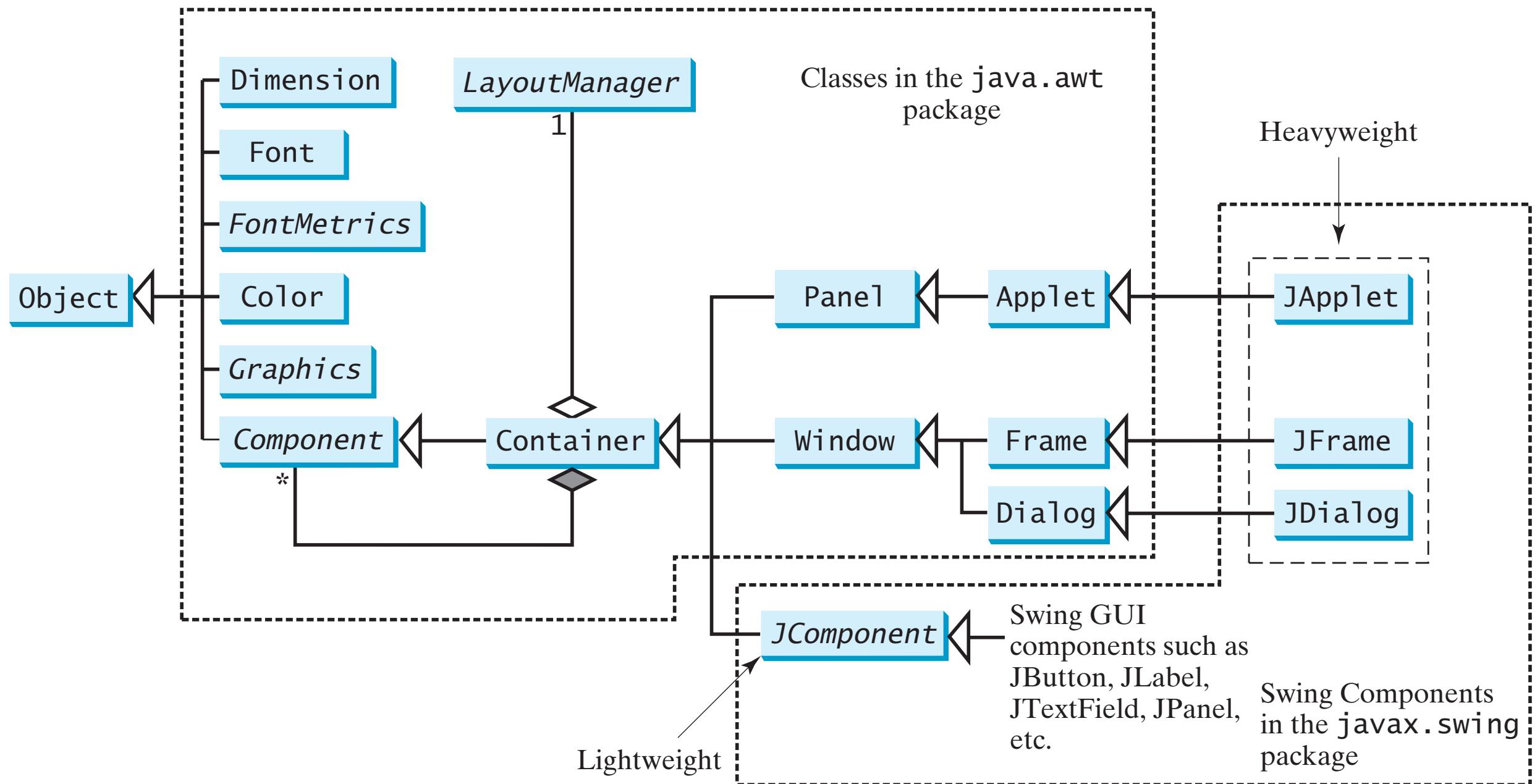
Features of the Java Foundation Classes

- Swing GUI Components
- Pluggable Look-and-Feel Support
- Accessibility API
- Java 2D API
- Internationalization

Swing Features

GUI Principles

- Components: GUI building blocks.
 - Buttons, menus, sliders, etc.
- Layout: arranging components to form a usable GUI.
 - Using layout *managers*.
- Events: reacting to user input.
 - Button presses, menu selections, etc.



Classes

`java.awt.Container`

`javax.swing.JFrame`

`javax.swing.JPanel`

`javax.swing.JApplet`

`javax.swing.JDialog`

`java.awt.Graphics`

`java.awt.Color`

`java.awt.Font`

`java.awt.FontMetrics`

`java.awt.Dimension`

`java.awt.LayoutManager`

JFrame

- A Frame is a top-level window with a title and a border.
- A frame, implemented as an instance of the JFrame class, is a window that typically has decorations such as a border, a title, and buttons for closing and iconifying the window.
Applications with a GUI typically use at least one frame.

Creating a frame

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ImageViewer
{
    private JFrame frame;

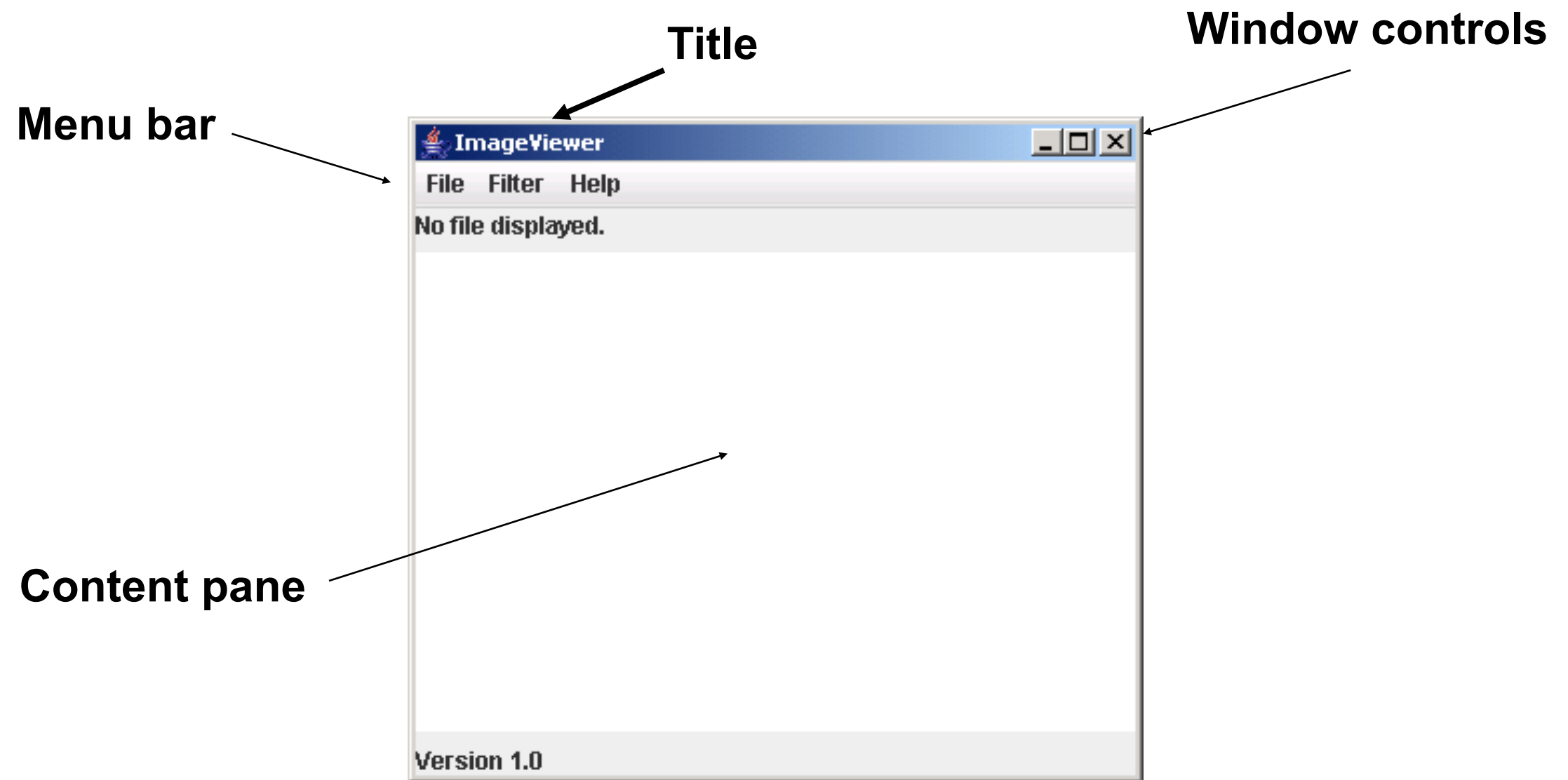
    /**
     * Create an ImageViewer show it on screen.
     */
    public ImageViewer()
    {
        makeFrame();
    }

    // rest of class omitted.
}
```

code for JFrame

```
JFrame frame = new ScoreBoard();  
frame.pack();  
frame.setVisible(true);
```

Elements of a frame



JFrame

javax.swing.JFrame

```
+JFrame()  
+JFrame(title: String)  
+setSize(width: int, height: int): void  
+setLocation(x: int, y: int): void  
+setVisible(visible: boolean): void  
+setDefaultCloseOperation(mode: int): void  
+setLocationRelativeTo(c: Component):  
    void  
+pack(): void
```

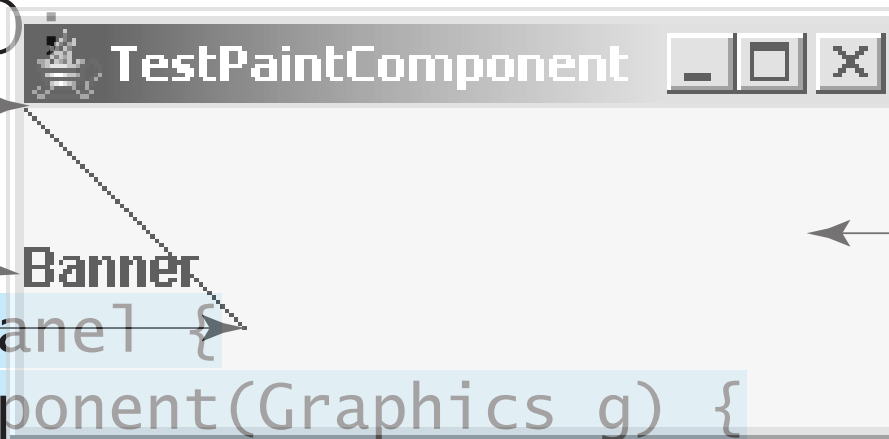
- JFrame is a top-level container to hold GUI components.

Graphics

Directly Render

```
1 import javax.swing.*;
2 import java.awt.Graphics;
3
4 public class TestPaintComponent extends JFrame {
5     public TestPaintComponent() {
6         add(new JPanel());
7     }
8
9     public static void main(String[] args) {
10         TestPaintComponent frame = new TestPaintComponent();
11         frame.setTitle("TestPaintComponent");
12         frame.setSize(200, 100);
13         frame.setLocationRelativeTo(null); // Center the frame
14         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
15         frame.setVisible(true);
16     }
17 }
```

```
19 class JPanel extends JComponent {
20     protected void paintComponent(Graphics g) {
21         super.paintComponent(g);
22         g.drawLine(0, 0, 50, 50);
23         g.drawString("Banner", 0, 40);
24     }
25 }
```



← This is a JPanel
object placed
inside a frame

TestPaintComponent.java

The Graphics

- The Graphics class contains the methods for drawing strings and shapes.

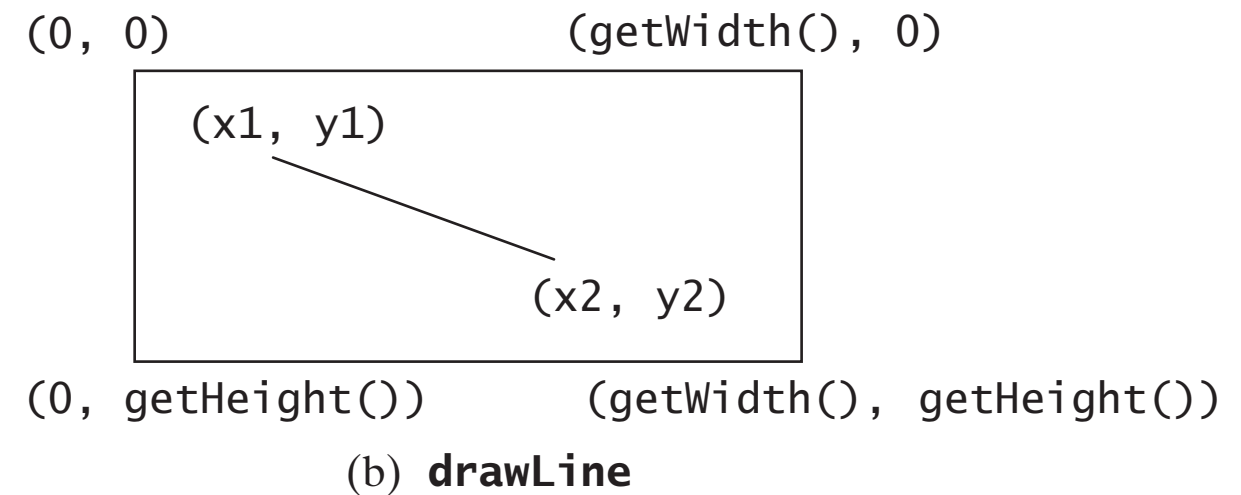
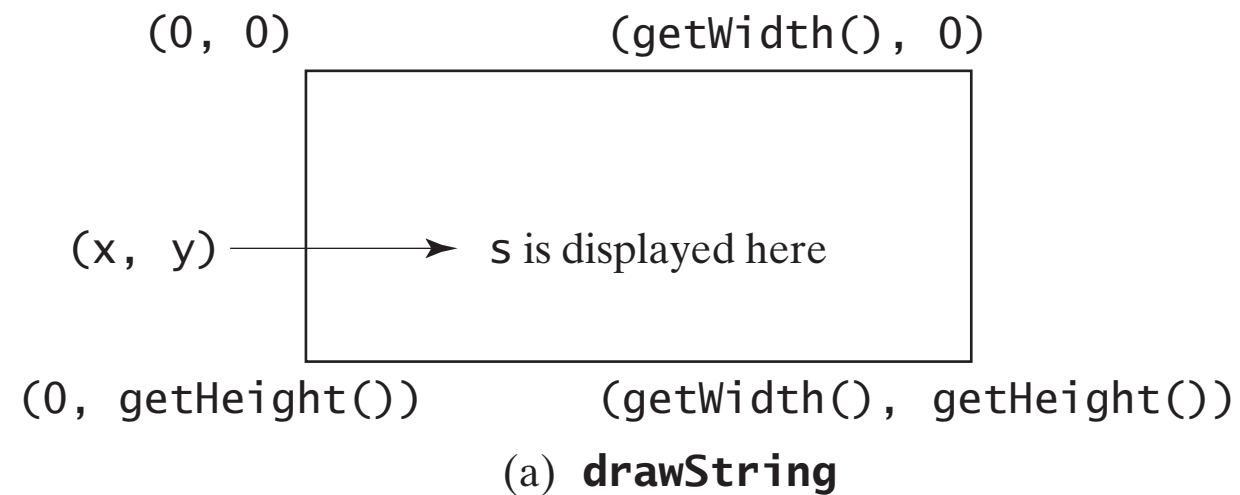
java.awt.Graphics

```
+setColor(color: Color): void
+setFont(font: Font): void
+drawString(s: String, x: int, y: int): void
+drawLine(x1: int, y1: int, x2: int, y2:
    int): void
+drawRect(x: int, y: int, w: int, h: int):
    void
+fillRect(x: int, y: int, w: int, h: int): void

+drawRoundRect(x: int, y: int, w: int, h: int, aw:
    int, ah: int): void
+fillRoundRect(x: int, y: int, w: int, h: int,
    aw: int, ah: int): void
+draw3DRect(x: int, y: int, w: int, h: int,
    raised: boolean): void
+fill3DRect(x: int, y: int, w: int, h: int,
    raised: boolean): void
+drawOval(x: int, y: int, w: int, h: int):
    void
+fillOval(x: int, y: int, w: int, h: int): void

+drawArc(x: int, y: int, w: int, h: int,
    startAngle: int, arcAngle: int): void
+fillArc(x: int, y: int, w: int, h: int,
    startAngle: int, arcAngle: int): void
+drawPolygon(xPoints: int[], yPoints:
    int[], nPoints: int): void
+fillPolygon(xPoints: int[], yPoints: int[],
    nPoints: int): void
+drawPolygon(g: Polygon): void
+fillPolygon(g: Polygon): void
+drawPolyline(xPoints: int[], yPoints:
    int[], nPoints: int): void
```

String and Line

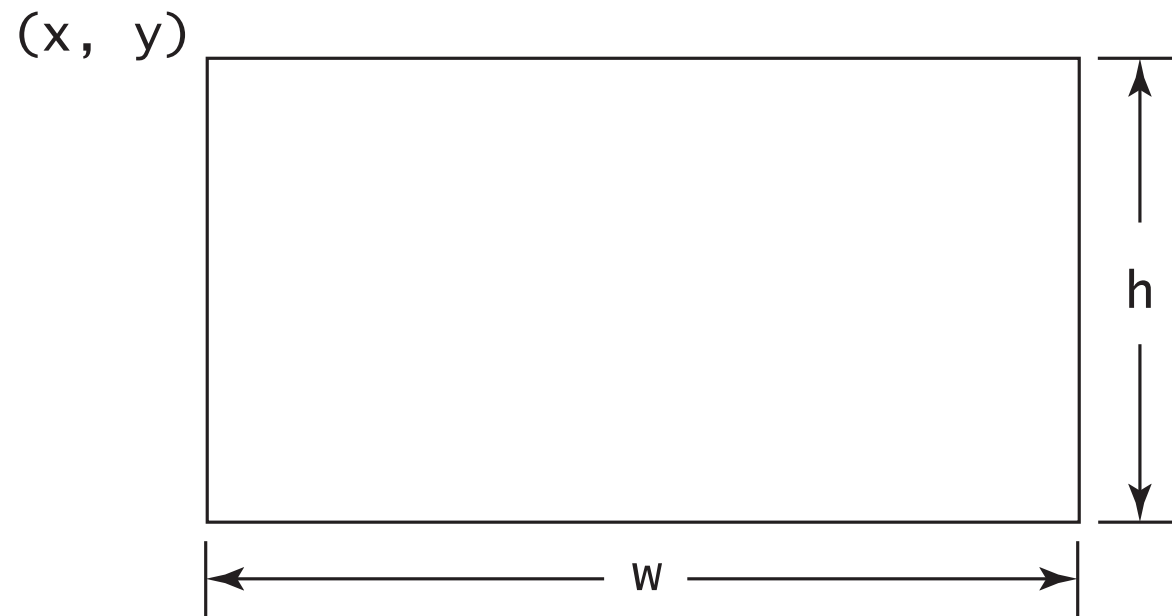


- **drawString(s, x, y)**
- **drawLine(x1, y1, x2, y2)**

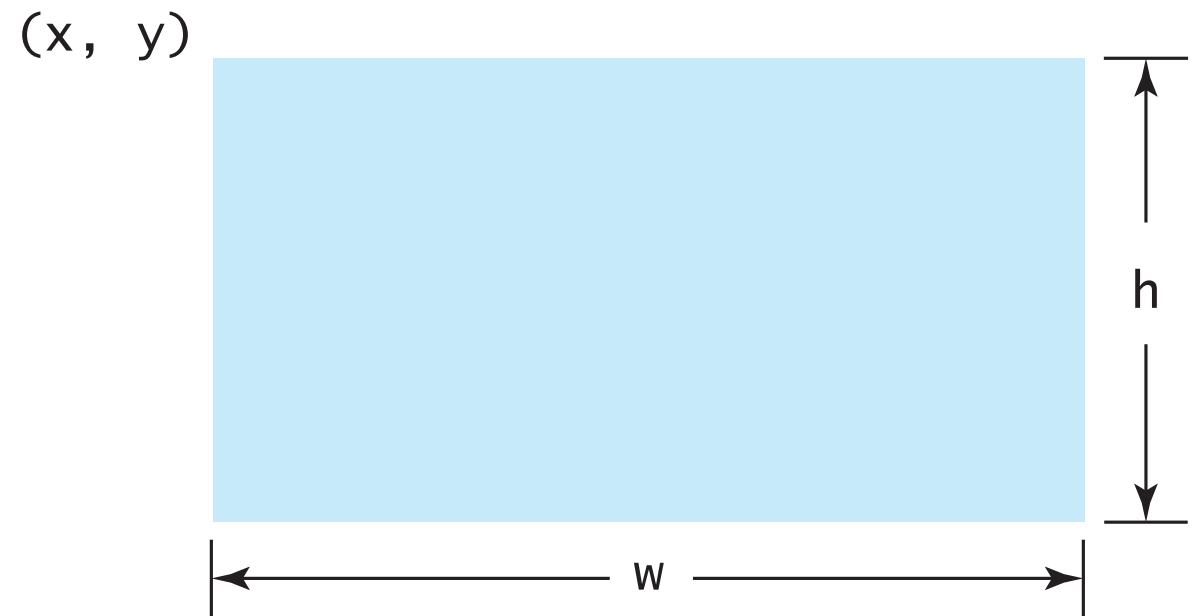
Color

- `public Color(int r, int g, int b);`
- `setColor(Color color);`
- `JButton jbtOK = new JButton("OK");`
`jbtOK.setBackground(color);`
`jbtOK.setForeground(new Color(100, 1, 1));`
- `jbtOK.setForeground(Color.RED);`

Draw and Fill Rectangle



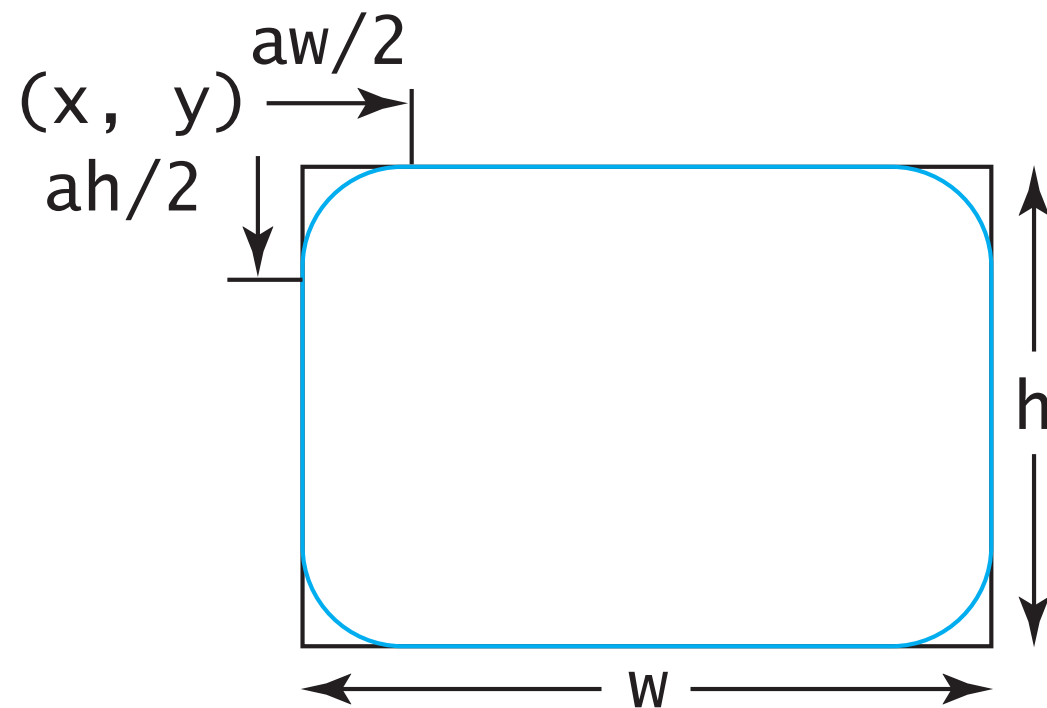
(a) Plain rectangle



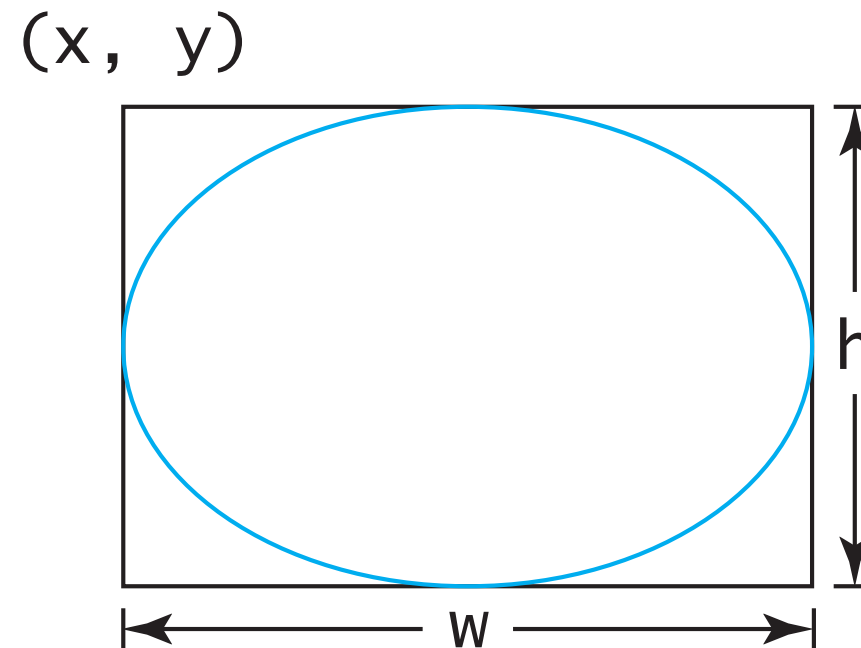
(b) Filled rectangle

- `drawRect(x, y, w, h)`
- `fillRect(x, y, w, h)`

RoundRect and Oval



(a) **drawRoundRect**

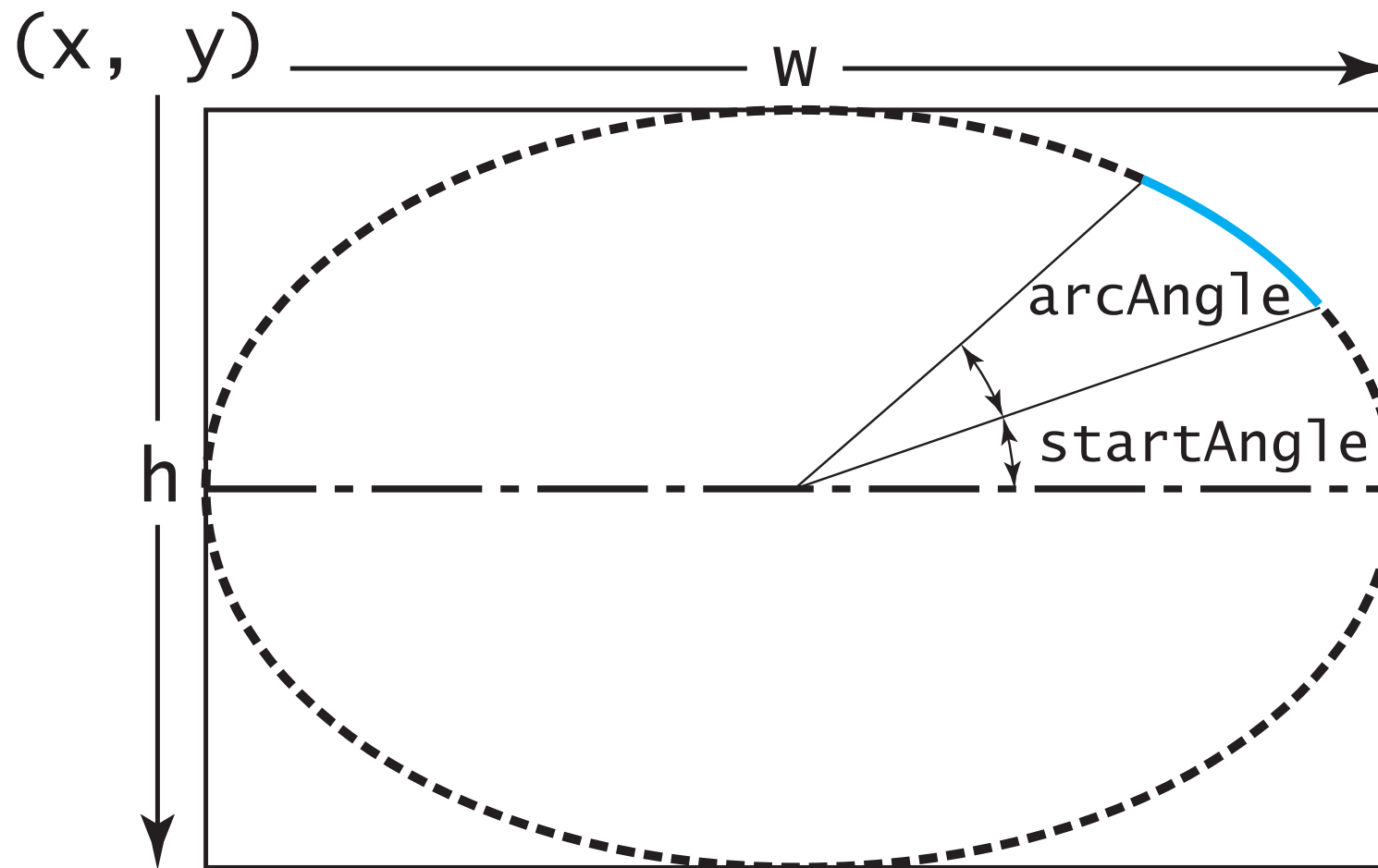


(b) **drawOval**

- **drawRoundRect**(x, y, w, h, aw, ah)
- **drawOval**(x, y, w, h)

FigurePanel.java
TestFigurePanel.java

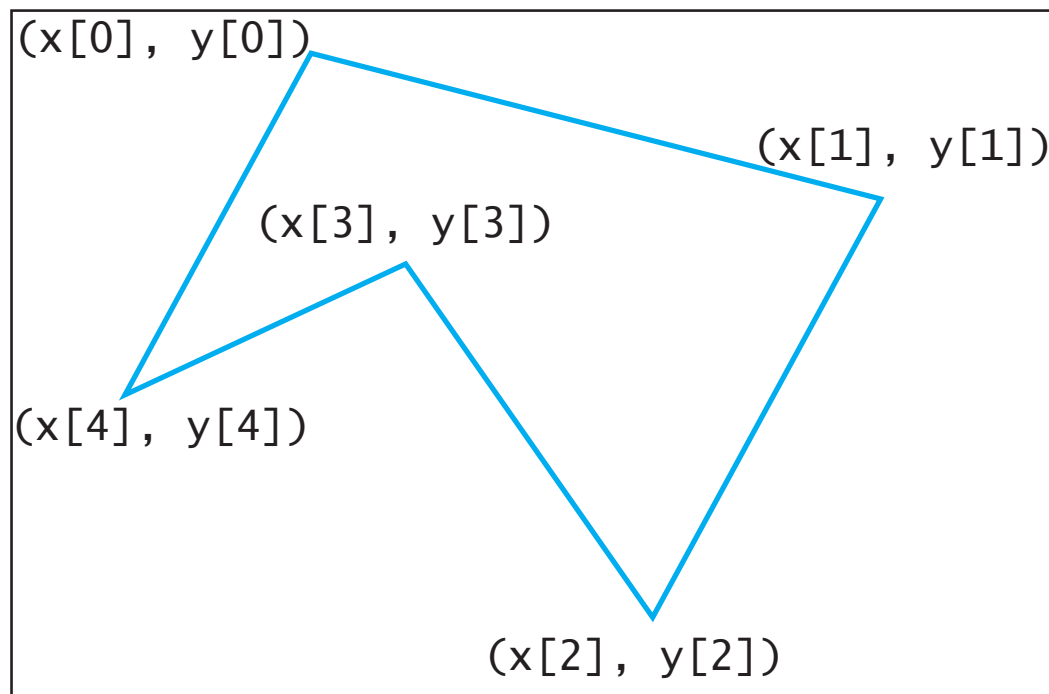
Arcs



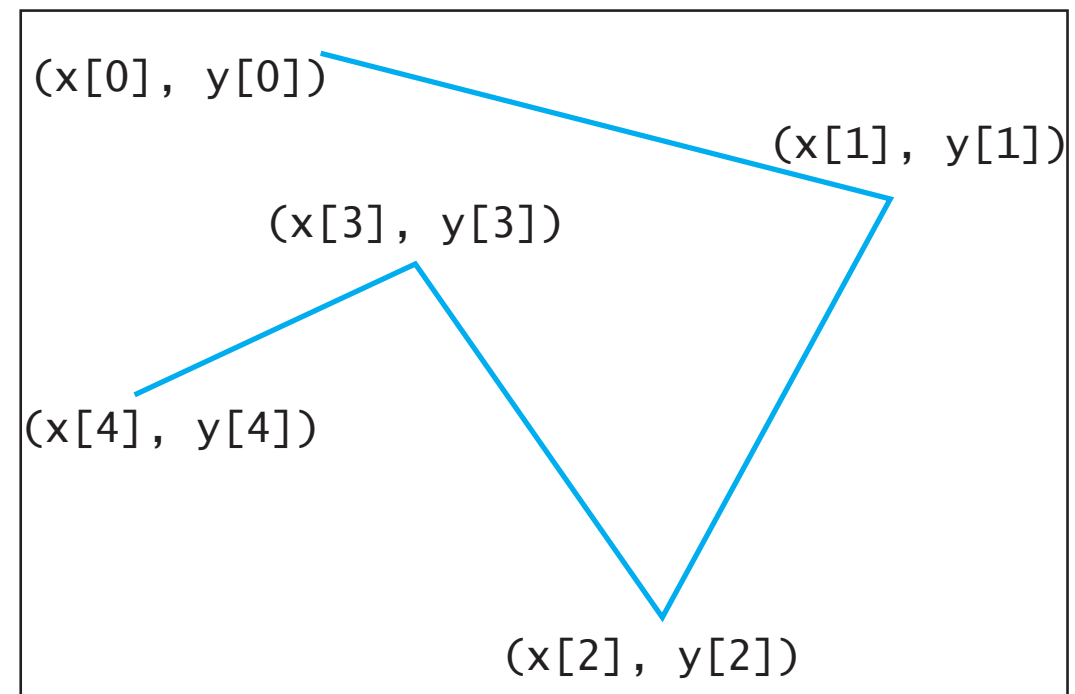
- `drawArc(int x, int y, int w, int h, int startAngle, int arcAngle);`
- `fillArc(int x, int y, int w, int h, int startAngle, int arcAngle);`

DrawArcs.java

Polygon and Polyline



(a) Polygon



(b) Polyline

- `Polygon polygon = new Polygon();`
`polygon.addPoint(40, 20);`
- `drawPolygon(Polygon polygon);`
`fillPolygon(Polygon polygon);`

Another way polygon

```
int x[] = {40, 70, 60, 45, 20};  
int y[] = {20, 40, 80, 45, 60};  
g.drawPolygon(x, y, x.length);  
g.drawPolyline(x, y, x.length);
```

英文字体从单字上分两种，一种有衬线（serif），一种无衬线（sanserif）。从26个字母比较上又分两种，一种自然宽度，一种等宽（monospace）。衬线字体笔画有粗细变化，且首尾带装饰线，无衬线字体笔画粗细均匀无变化，也没有装饰线。自然宽度字体 m 最宽，i 最窄，等宽字体26个字母全部一样宽。

... a style from Font.PLAIN (0), Font.BOLD (1), Font.ITALIC (2), and Font.BOLD + Font.ITALIC (3),

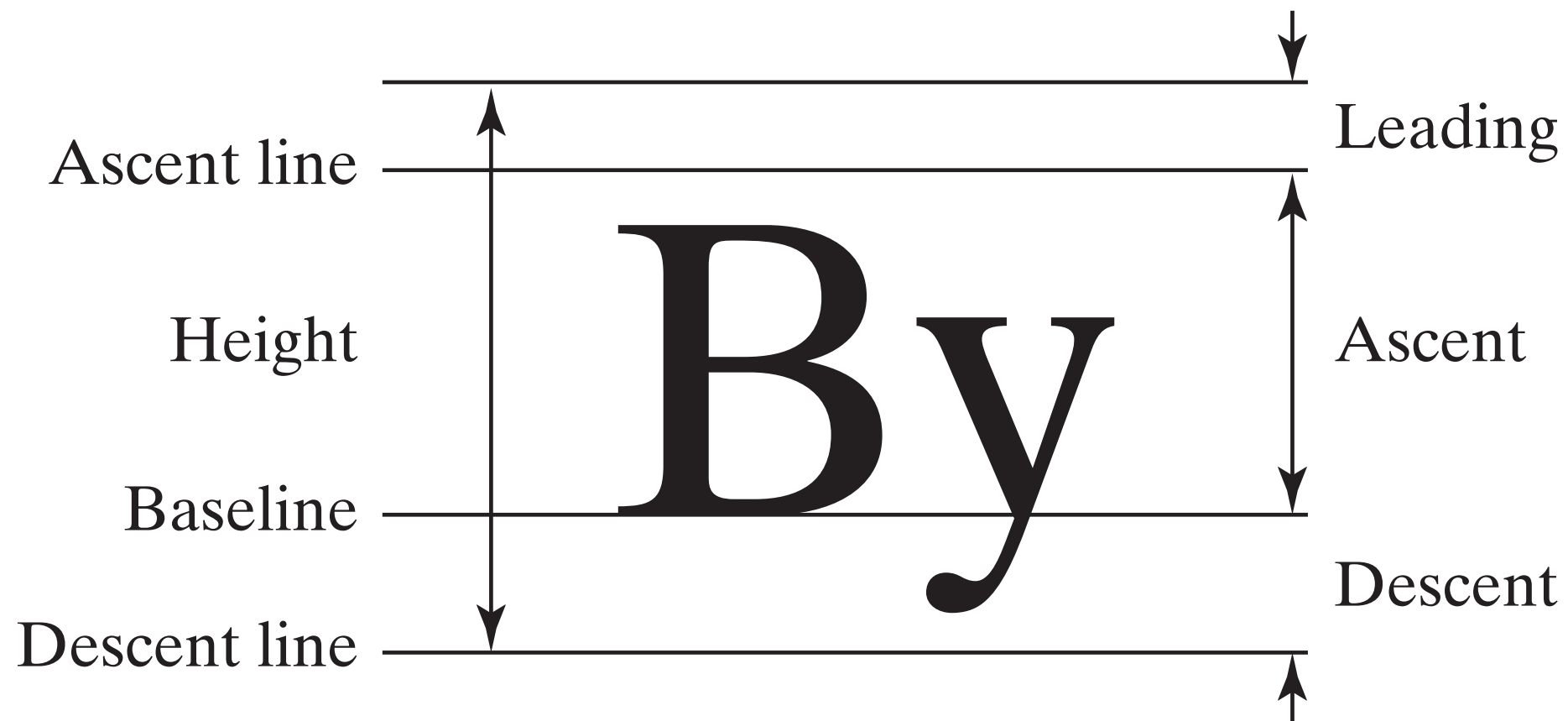
- and specify a font size of any positive integer.

- `Font font1 = new Font("SansSerif", Font.BOLD, 16);`
`Font font2 = new Font("Serif", Font.BOLD + Font.ITALIC, 12);`
- `JButton jbtOK = new JButton("OK");`
`jbtOK.setFont(font1);`

Enum Fonts

```
GraphicsEnvironment e =  
GraphicsEnvironment.getLocalGraphicsEnvironment();  
String[] fontnames = e.getAvailableFontFamilyNames();  
for (int i = 0; i < fontnames.length; i++)  
    System.out.println(fontnames[i]);
```

FontMetrics



- In Graphics:

- `FontMetrics getFontMetrics(Font font)`

- `FontMetrics getFontMetrics()`

`TestCenterMessage.java`

FontMetrics

- `public int getAscent() // Return the ascent`
- `public int getDescent() // Return the descent`
- `public int stringAscent(String str) // Return the ascent of the string`
- `public int stringWidth(String str) // Return the width of the string`
- `public int getHeight() // Return the height`

This is a MessagePanel object

`(xCoordinate, yCoordinate)`

```
kCoordinate = getWidth / 2 - stringWidth / 2;  
yCoordinate = getHeight / 2 - stringAscent / 2;
```

MessagePanel.java
TestMessagePanel.java

Images as Icons

- `ImageIcon icon = new ImageIcon("image/us.gif");
JLabel lblImage = new JLabel(icon);`
- `Image image = icon.getImage();`
- `g.drawImage(image, 0, 0, getWidth(), getHeight(),
this);`

java.awt.Graphics

```
+drawImage(image: Image, x: int, y: int,  
           bgcolor: Color, observer:  
           ImageObserver): void  
  
+drawImage(image: Image, x: int, y: int,  
           observer: ImageObserver): void  
  
+drawImage(image: Image, x: int, y: int,  
           width: int, height: int, observer:  
           ImageObserver): void  
  
+drawImage(image: Image, x: int, y: int,  
           width: int, height: int, bgcolor: Color,  
           observer: ImageObserver): void
```

- JPanel is a kind of ImageObserver

DisplayImage.java

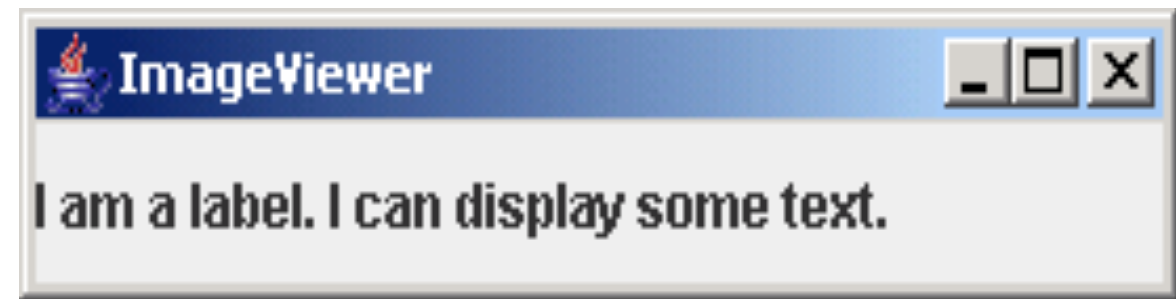
LayoutManager

The content pane

```
/**
 * Create the Swing frame and its content.
 */
private void makeFrame()
{
    frame = new JFrame("ImageViewer");
    Container contentPane = frame.getContentPane();

    JLabel label = new JLabel("I am a label.");
    contentPane.add(label);

    frame.pack();
    frame.setVisible(true);
}
```



content-pane delegation.

Adding Components

```
1 import javax.swing.*;      MyFrameWithComponents.java
2
3 public class MyFrameWithComponents {
4     public static void main(String[] args) {
5         JFrame frame = new JFrame("MyFrameWithComponents");
6
7         // Add a button into the frame
8         JButton jbtOK = new JButton("OK");
9         frame.add(jbtOK);
10
11         frame.setSize(400, 300);
12         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13         frame.setLocationRelativeTo(null); // Center the frame
14         frame.setVisible(true);
15     }
16 }
17
18 container.remove(jbtOK);
```

Layout Manager

- The layout manager works behind the scenes to place components in the correct locations.

```
LayoutManager layoutManager = new XLayout();  
container.setLayout(layoutManager);
```

FlowLayout

- The components are arranged in the container from left to right in the order in which they were added. When one row is filled, a new row is started.
- You can specify the way the components are aligned by using one of three constants: `FlowLayout.RIGHT`, `FlowLayout.CENTER`, or `FlowLayout.LEFT`. You can also specify the gap between components in pixels.

FlowLayout

java.awt.FlowLayout

-alignment: int
-hgap: int
-vgap: int

+FlowLayout()
+FlowLayout(alignment: int)
+FlowLayout(alignment: int, hgap: int, vgap: int)

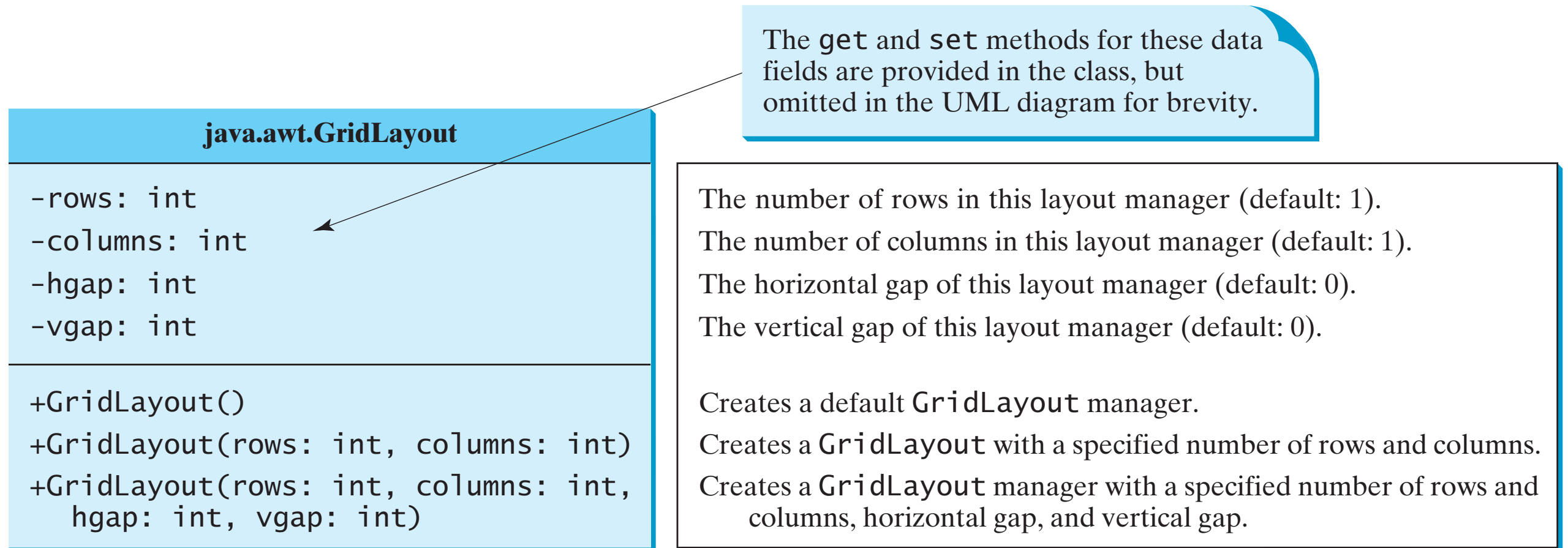
The `get` and `set` methods for these data fields are provided in the class, but omitted in the UML diagram for brevity.

The alignment of this layout manager (default: CENTER).
The horizontal gap of this layout manager (default: 5 pixels).
The vertical gap of this layout manager (default: 5 pixels).

Creates a default FlowLayout manager.
Creates a FlowLayout manager with a specified alignment.
Creates a FlowLayout manager with a specified alignment, horizontal gap, and vertical gap.

ShowFlowLayout.java

GridLayout



- The components are placed in the grid from left to right, starting with the first row, then the second, and so on, in the order in which they are added.

ShowGridLayout.java

BorderLayout



```
add(Component, index)
BorderLayout.EAST,
BorderLayout.SOUTH,
BorderLayout.WEST,
BorderLayout.NORTH,
BorderLayout.CENTER
```

The `get` and `set` methods for these data fields are provided in the class, but omitted in the UML diagram for brevity.

java.awt.BorderLayout

-hgap: int
-vgap: int

+BorderLayout()
+BorderLayout(hgap: int, vgap: int)

The horizontal gap of this layout manager (default: 0).
The vertical gap of this layout manager (default: 0).

Creates a default BorderLayout manager.

Creates a BorderLayout manager with a specified number of horizontal gap, and vertical gap.

ShowBorderLayout.java

Properties of Layout Managers

// Create a layout manager

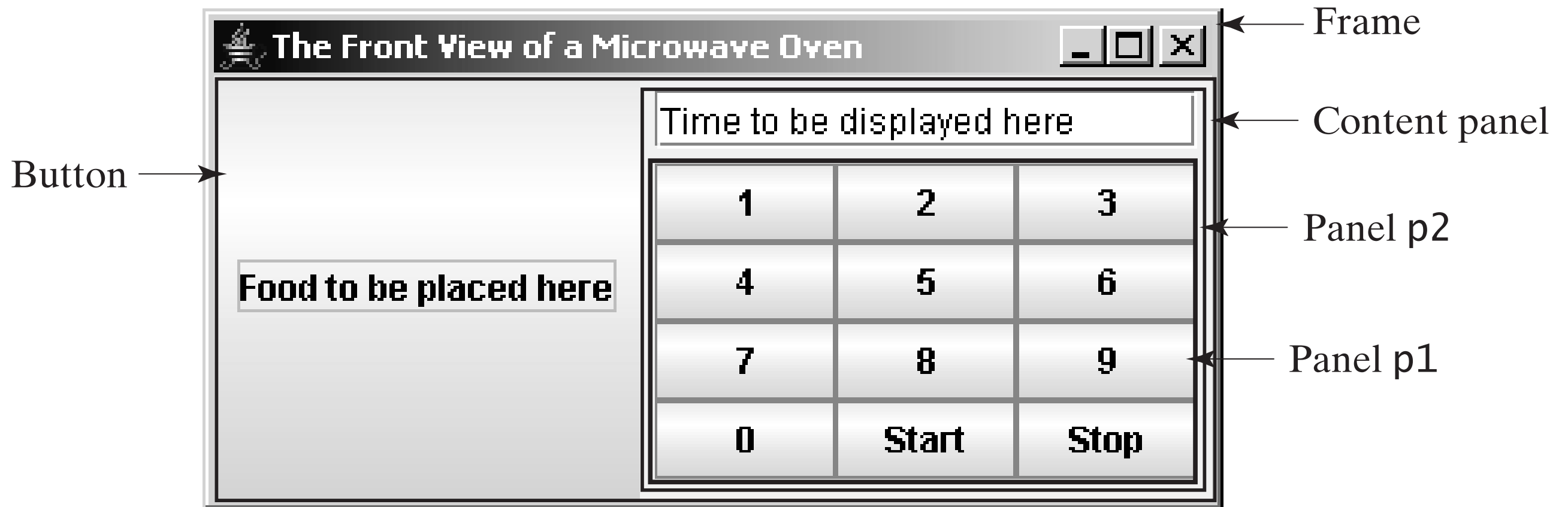
```
FlowLayout flowLayout = new FlowLayout();
```

// Set layout properties

```
flowLayout.setAlignment(FlowLayout.RIGHT);
```

```
flowLayout.setHgap(10); flowLayout.setVgap(20);
```

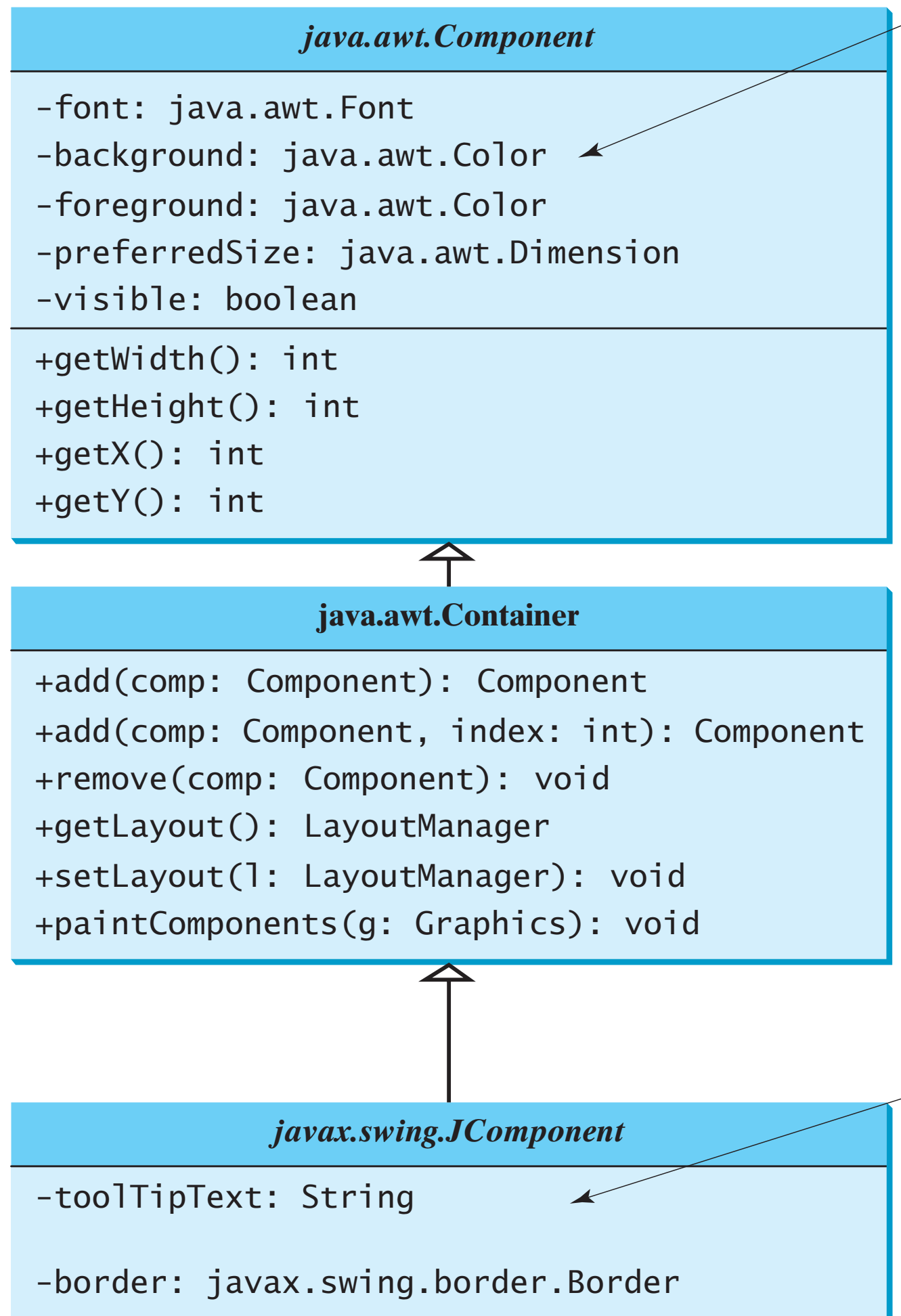
Using Panels as Subcontainers



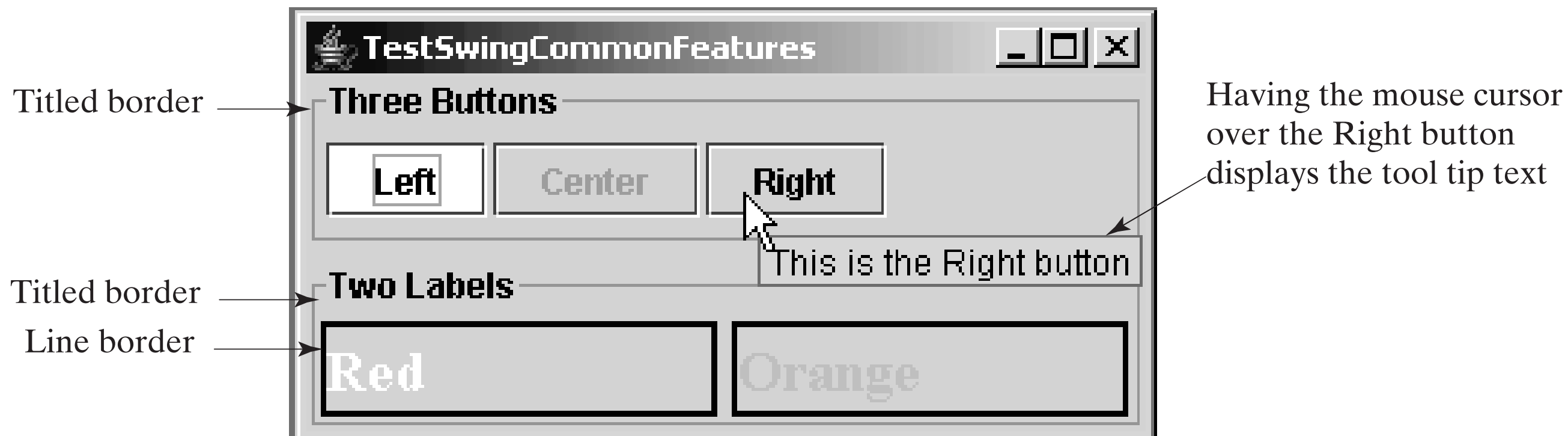
- Panels can be placed inside a frame or inside another panel.

TestPanels.java

Common Features of Swing GUI Components

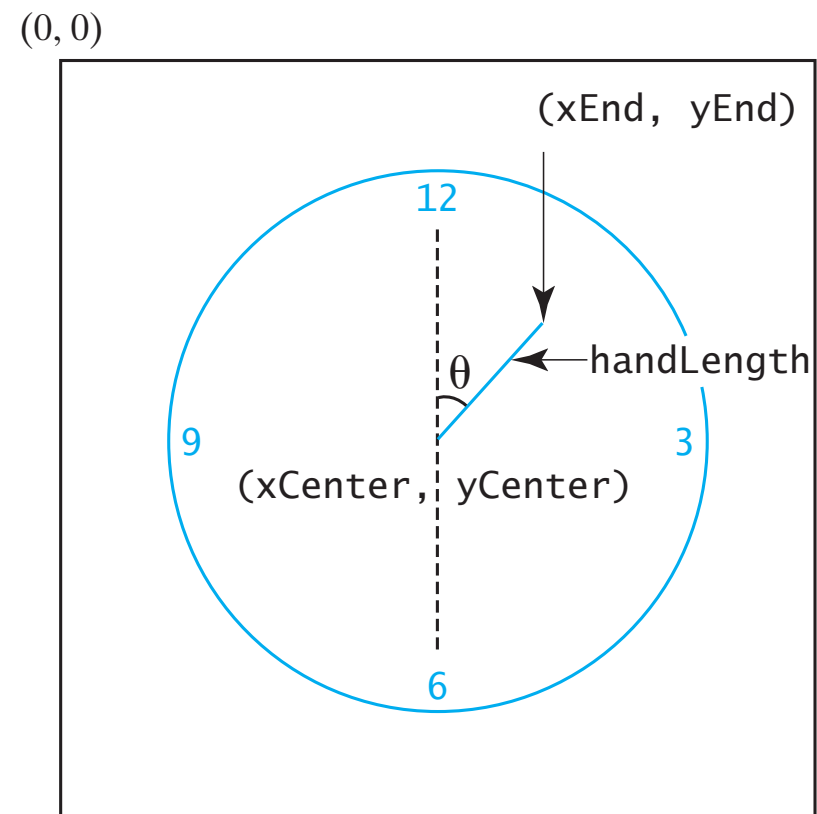


What are they



TestSwingCommonFeatures.java

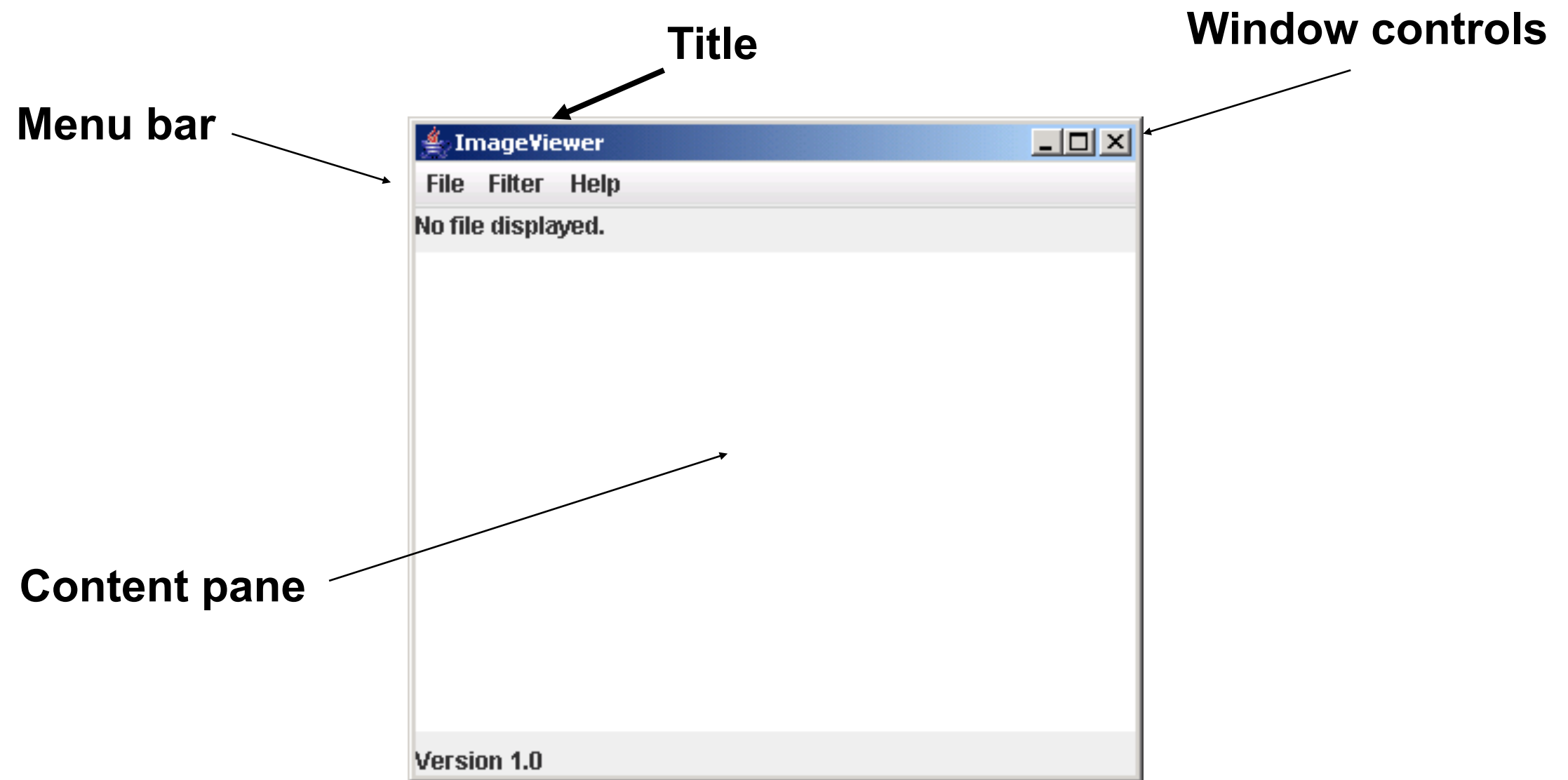
StillClock



DisplayClock.java
StillClock.java

Frame

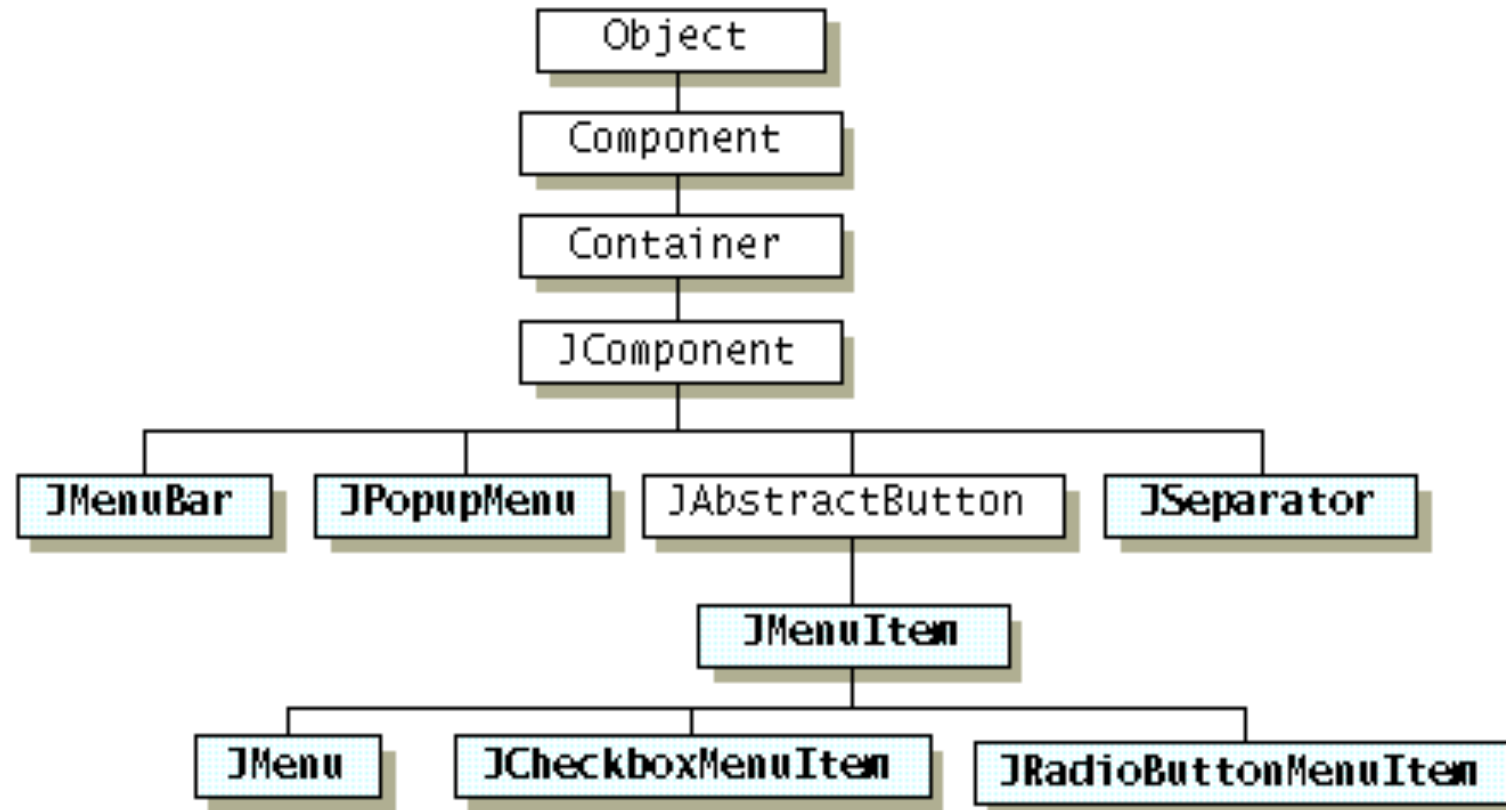
Elements of a frame



Adding menus

- `JMenuBar`
 - Displayed below the title.
 - Contains the menus.
- `JMenu`
 - e.g. *File*. Contains the menu items.
- `JMenuItem`
 - e.g. *Open*. Individual items.

The Menu



```
private void makeMenuBar(JFrame frame)
{
    JMenuBar menubar = new JMenuBar();
    frame.setJMenuBar(menubar);

    // create the File menu
    JMenu fileMenu = new JMenu("File");
    menubar.add(fileMenu);

    JMenuItem openItem = new JMenuItem("Open");
    fileMenu.add(openItem);

    JMenuItem quitItem = new JMenuItem("Quit");
    fileMenu.add(quitItem);
}
```

To create menus

- Create a JMenuBar, set it to the JFrame
- Create some JMenus, add them to the JMenuBar
- Create some JMenuItems, add them to the JMenus
- Add ActionListener to every JMenuItem to receive the event

Event handling

- Events correspond to user interactions with components.
- Components are associated with different event types.
 - Frames are associated with **WindowEvent**.
 - Menus are associated with **ActionEvent**.
- Objects can be notified when an event occurs.
 - Such objects are called *listeners*.

Centralized event receipt

- A single object handles all events.
 - Implements the **ActionListener** interface.
 - Defines an **actionPerformed** method.
- It registers as a listener with each component.
 - **item.addActionListener(this)**
- It has to work out which component has dispatched the event.

```

public class ImageViewer implements ActionListener
{
    ...
    public void actionPerformed(ActionEvent e)
    {
        String command = e.getActionCommand();
        if(command.equals("Open")) {
            ...
        }
        else if (command.equals("Quit")) {
            ...
        }
        ...
    }
    ...
    private void makeMenuBar(Jframe frame)
    {
        ...
        openItem.addActionListener(this);
        ...
    }
}

```

Centralized event handling

- The approach works.
- It is used, so you should be aware of it.
- However ...
 - It does not scale well.
 - Identifying components by their text is fragile.
- An alternative approach is preferred.

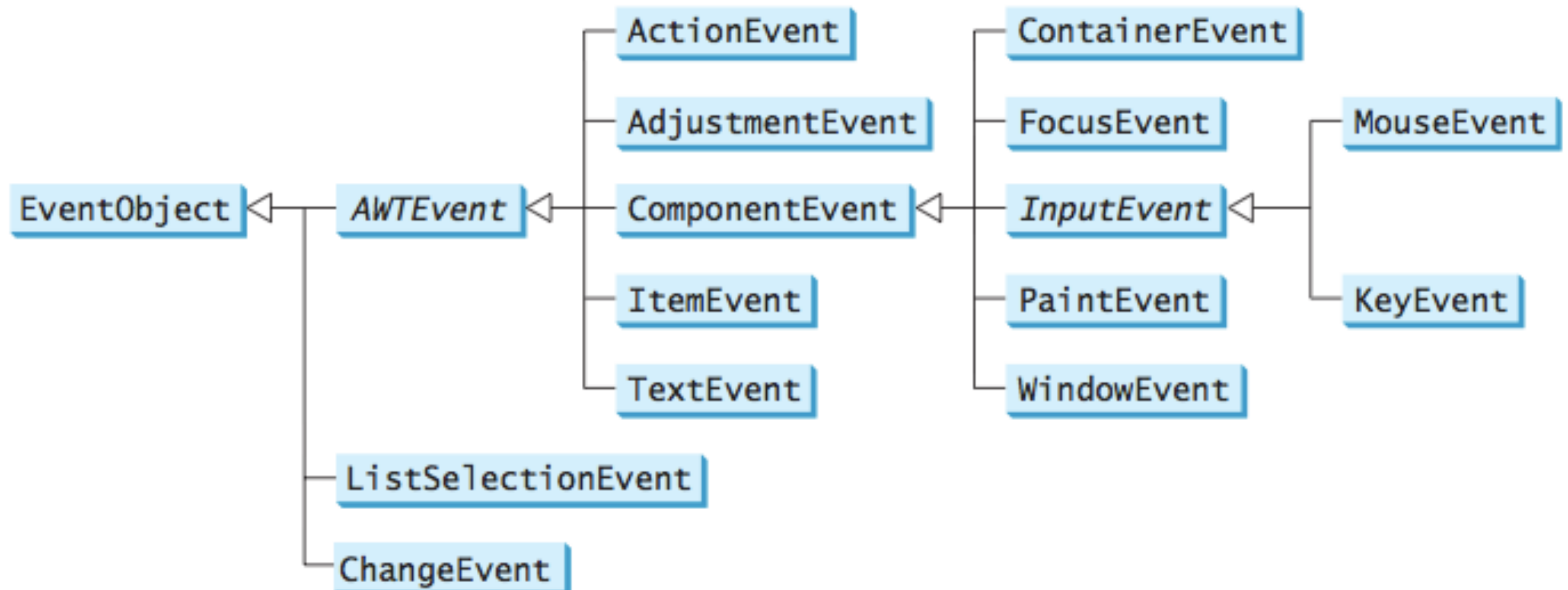
Anonymous action listener

```
JMenuItem openItem = new JMenuItem("Open");  
  
openItem.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        openFile();  
    }  
});
```

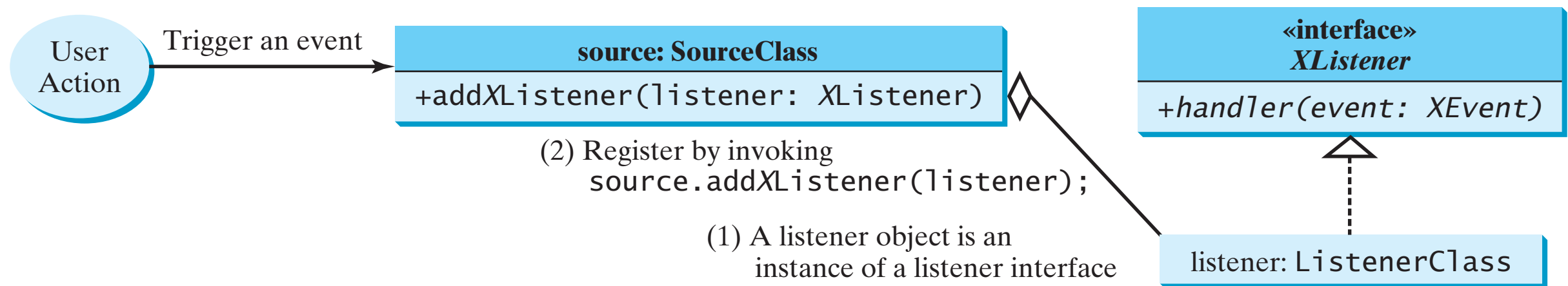
Anonymous action listener

```
JMenuItem openItem = new JMenuItem("Open");  
  
openItem.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        openFile();  
    }  
});
```

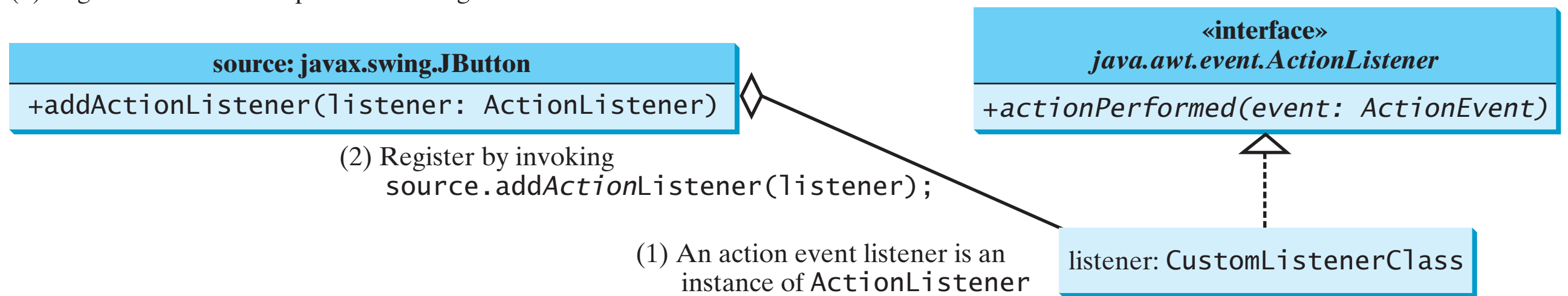
What Events We Have?



<i>User Action</i>	<i>Source Object</i>	<i>Event Type Fired</i>
Click a button	 JButton 	 ActionEvent
Press return on a text field	 JTextField 	 ActionEvent
Select a new item	 JComboBox 	 ItemEvent, ActionEvent
Select item(s)	 JList 	 ListSelectionEvent
Click a check box	 JCheckBox 	 ItemEvent, ActionEvent
Click a radio button	 JRadioButton 	 ItemEvent, ActionEvent
Select a menu item	 JMenuItem 	 ActionEvent
Move the scroll bar	 JScrollBar 	 AdjustmentEvent
Move the scroll bar	 JSlider 	 ChangeEvent
Window opened, closed, iconified, deiconified, or closing	 Window 	 WindowEvent
Mouse pressed, released, clicked, entered, or exited	 Component 	 MouseEvent
Mouse moved or dragged	 Component 	 MouseEvent
Key released or pressed	 Component 	 KeyEvent
Component added or removed from the container	 Container 	 ContainerEvent
Component moved, resized, hidden, or shown	 Component 	 ComponentEvent
Component gained or lost focus	 Component 	 FocusEvent



(a) A generic source component with a generic listener



(b) A JButton source component with an ActionListener

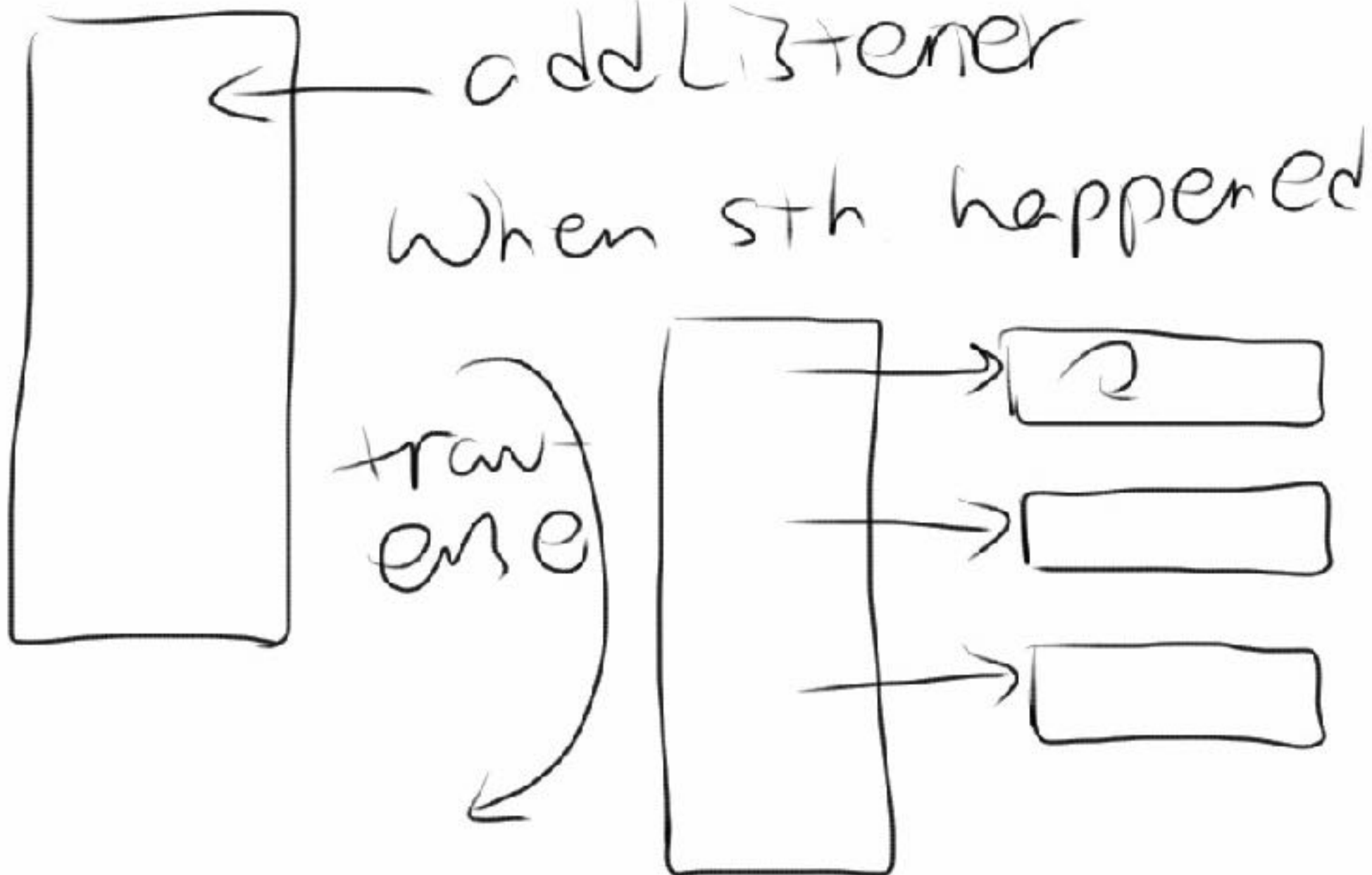
Dynamic Events

- There is more than one listener attached to each components.
- During the execution of the program, listeners are dynamically added and removed.

DynamicEvents.java

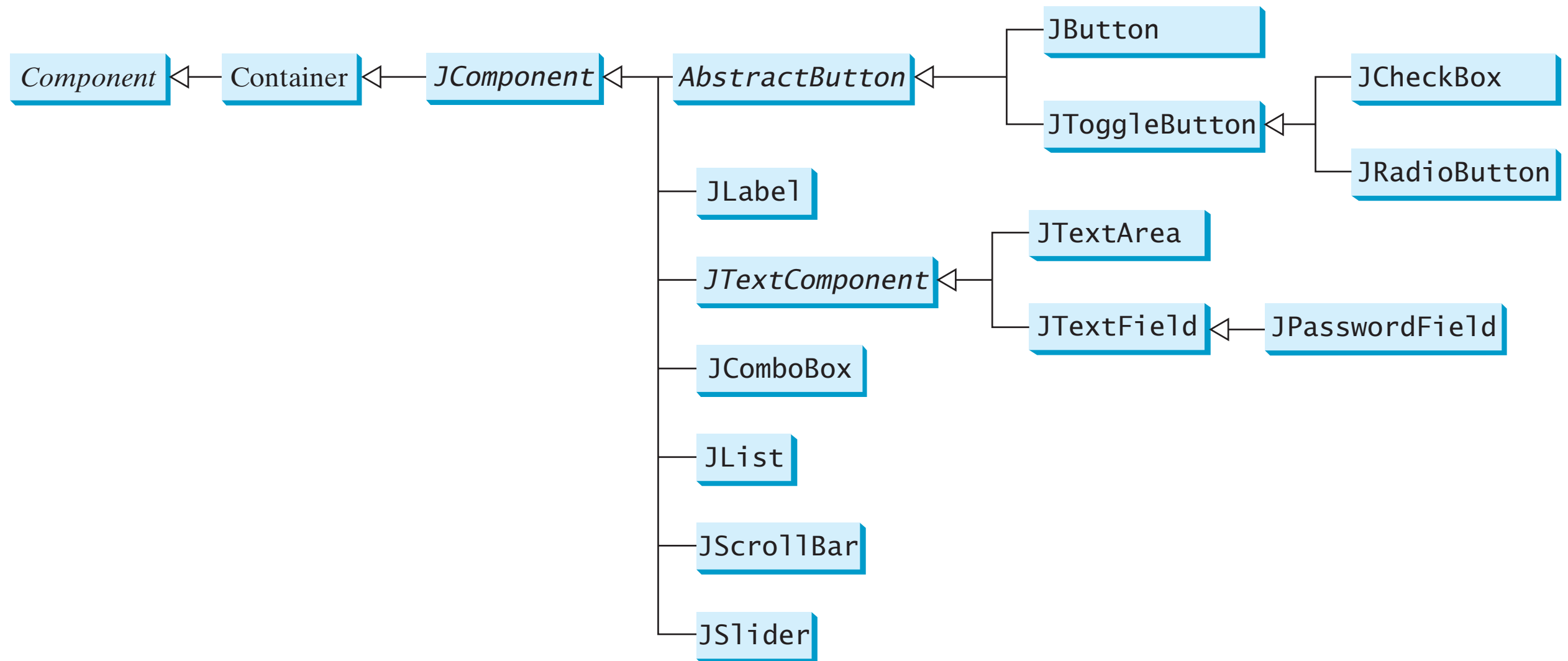
How Does It Work?

In the camp



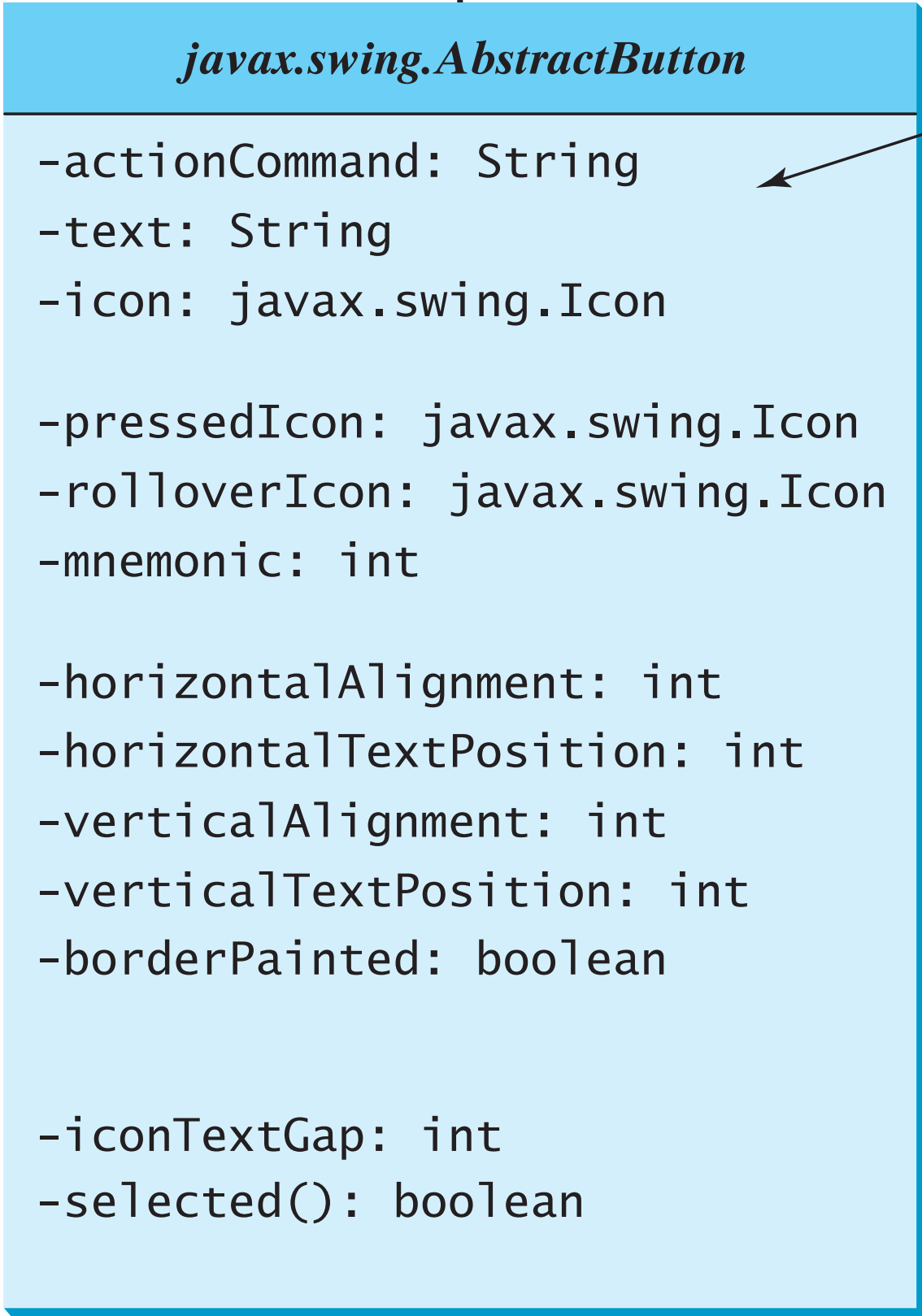
components and dialogs

Components



JButton

javax.swing.AbstractButton



- actionCommand: String
- text: String
- icon: javax.swing.Icon
- pressedIcon: javax.swing.Icon
- rolloverIcon: javax.swing.Icon
- mnemonic: int
- horizontalAlignment: int
- horizontalTextPosition: int
- verticalAlignment: int
- verticalTextPosition: int
- borderPainted: boolean
- iconTextGap: int
- selected(): boolean

- TestButtonIcons.java
- ButtonDemo.java

CheckBoxes

javax.swing.AbstractButton



javax.swing.JToggleButton



javax.swing.JCheckBox

```
+JCheckBox()  
+JCheckBox(text: String)  
+JCheckBox(text: String, selected:  
    boolean)  
+JCheckBox(icon: Icon)  
+JCheckBox(text: String, icon: Icon)  
+JCheckBox(text: String, icon: Icon,  
    selected: boolean)  
+addActionListener(listener:  
    ActionListener) : void  
+addItemListener(listener: ItemListener)  
    : void
```

- **CheckBoxDemo.java**

RadioButton

javax.swing.AbstractButton



javax.swing.JToggleButton



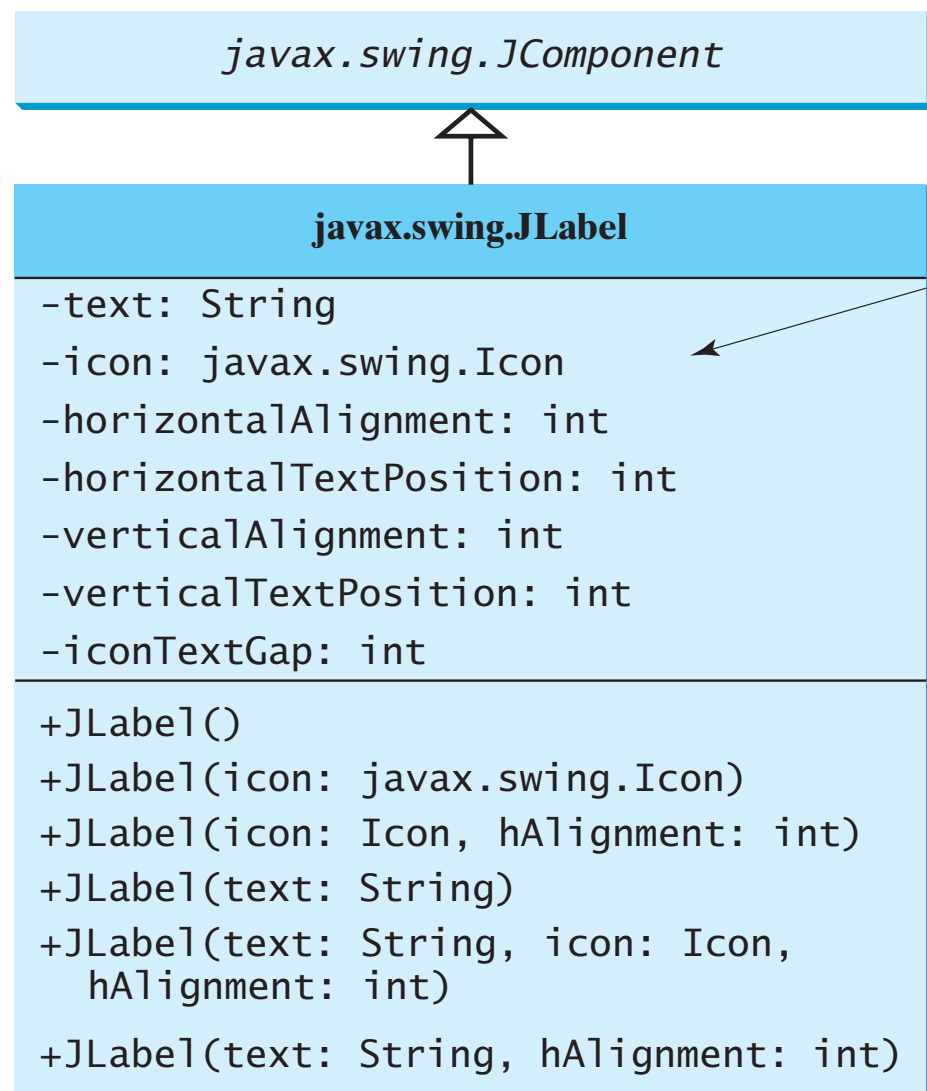
javax.swing.JRadioButton

```
+JRadioButton()  
+JRadioButton(text: String)  
+JRadioButton(text: String, selected:  
  boolean)  
+JRadioButton(icon: Icon)  
+JRadioButton(text: String, icon: Icon)  
+JRadioButton(text: String, icon: Icon,  
  selected: boolean)  
+addActionEvent(listener:  
  ActionListener): void  
+addItemListener(listener: ItemListener)  
  : void
```

JPanel with
GridLayout
for three
radio buttons



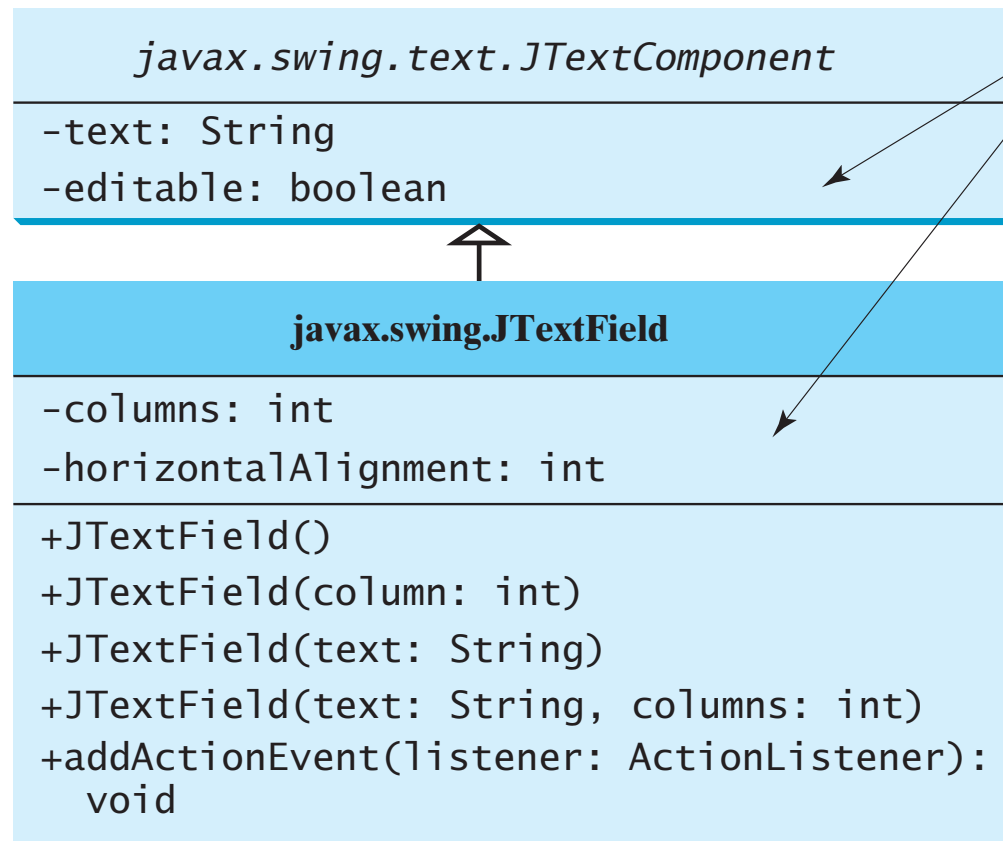
- **RadioButtonDemo.java**



Label

```
// Create an image icon from an image file
ImageIcon icon = new ImageIcon("image/grapes.gif");
// Create a label with a text, an icon,
// with centered horizontal alignment
JLabel jlbl = new JLabel("Grapes", icon, SwingConstants.CENTER);
//Set label's text alignment and gap between text and icon
jlbl.setHorizontalTextPosition(SwingConstants.CENTER);
jlbl.setVerticalTextPosition(SwingConstants.BOTTOM);
jlbl.setIconTextGap(5);
```

TextField



JPanel with
BorderLayout
for a label and
a text field

- TextFieldDemo.java

TextArea

javax.swing.text.JTextComponent



javax.swing.JTextArea

-columns: int
-rows: int
-tabSize: int
-lineWrap: boolean

-wrapStyleWord: boolean

+JTextArea()
+JTextArea(rows: int, columns: int)
+JTextArea(text: String)
+JTextArea(text: String, rows: int, columns: int)
+append(s: String): void
+insert(s: String, pos: int): void
+replaceRange(s: String, start: int, end: int): void
+getLineCount(): int

- DescriptionPanel.java
- TextAreaDemo.java

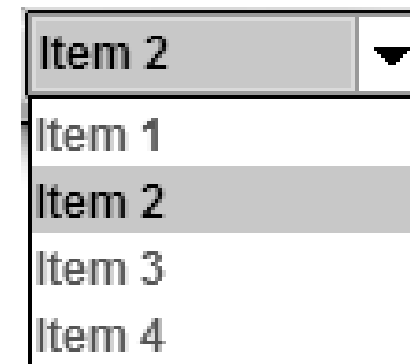
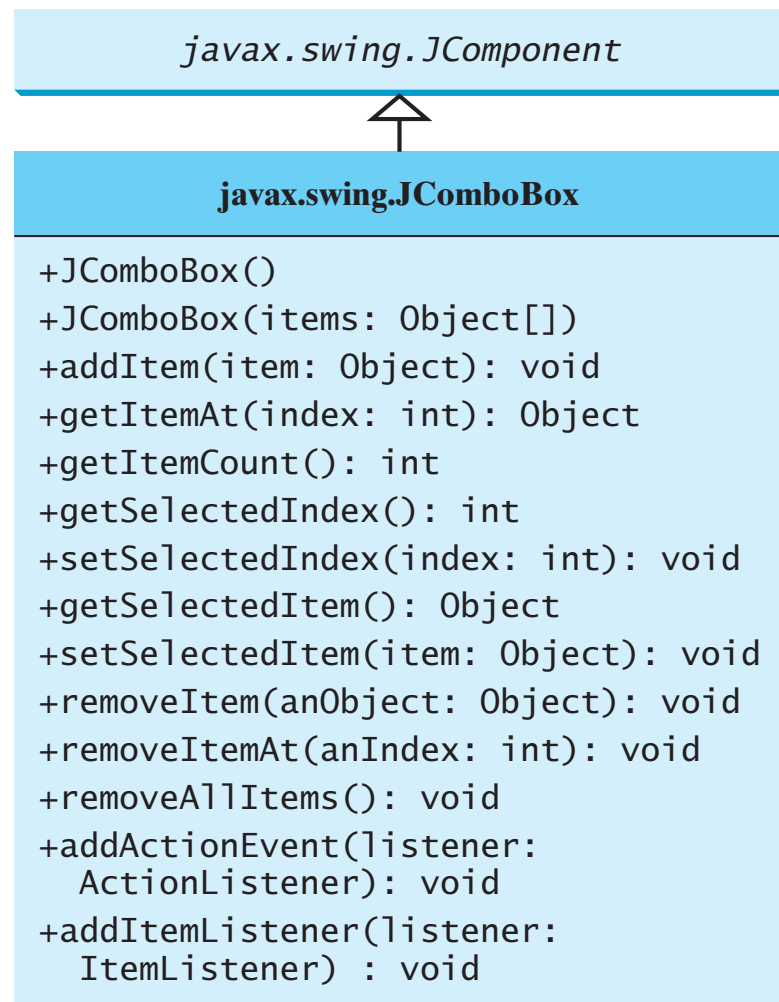
A label
showing an
image and a
text



DescriptionPanel
with BorderLayout

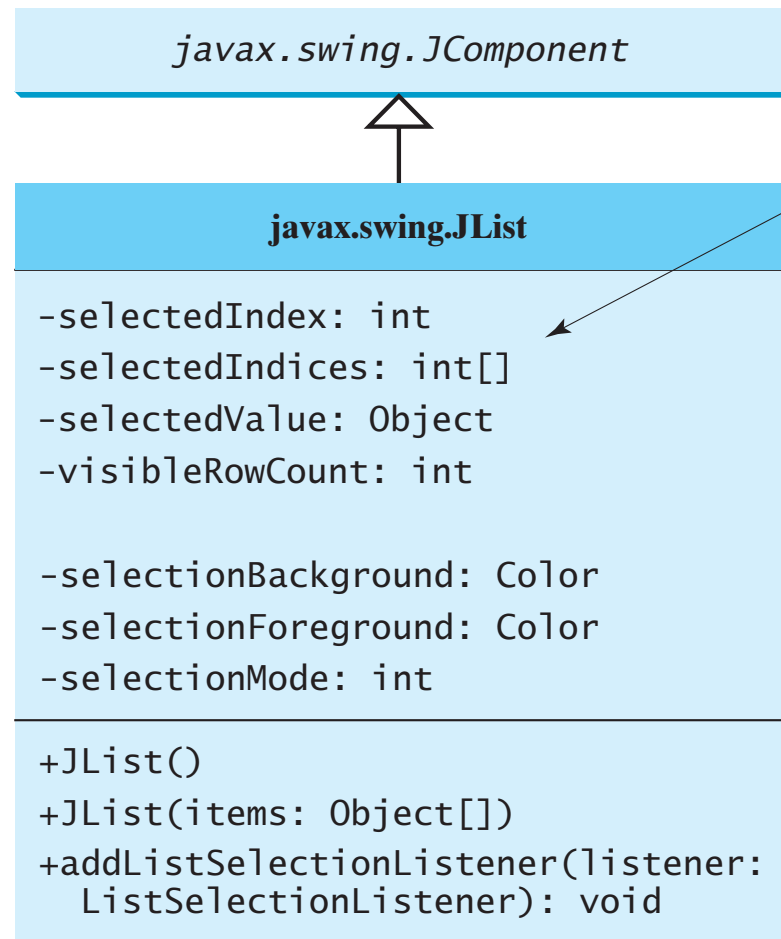
A text area
inside a
scroll panel

ComboBox



- **ComboBoxDemo.java**

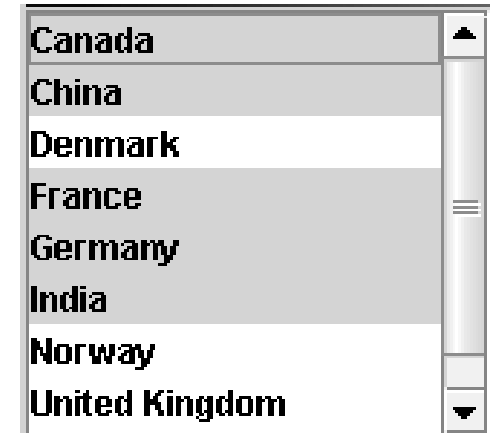
List



(a) Single selection



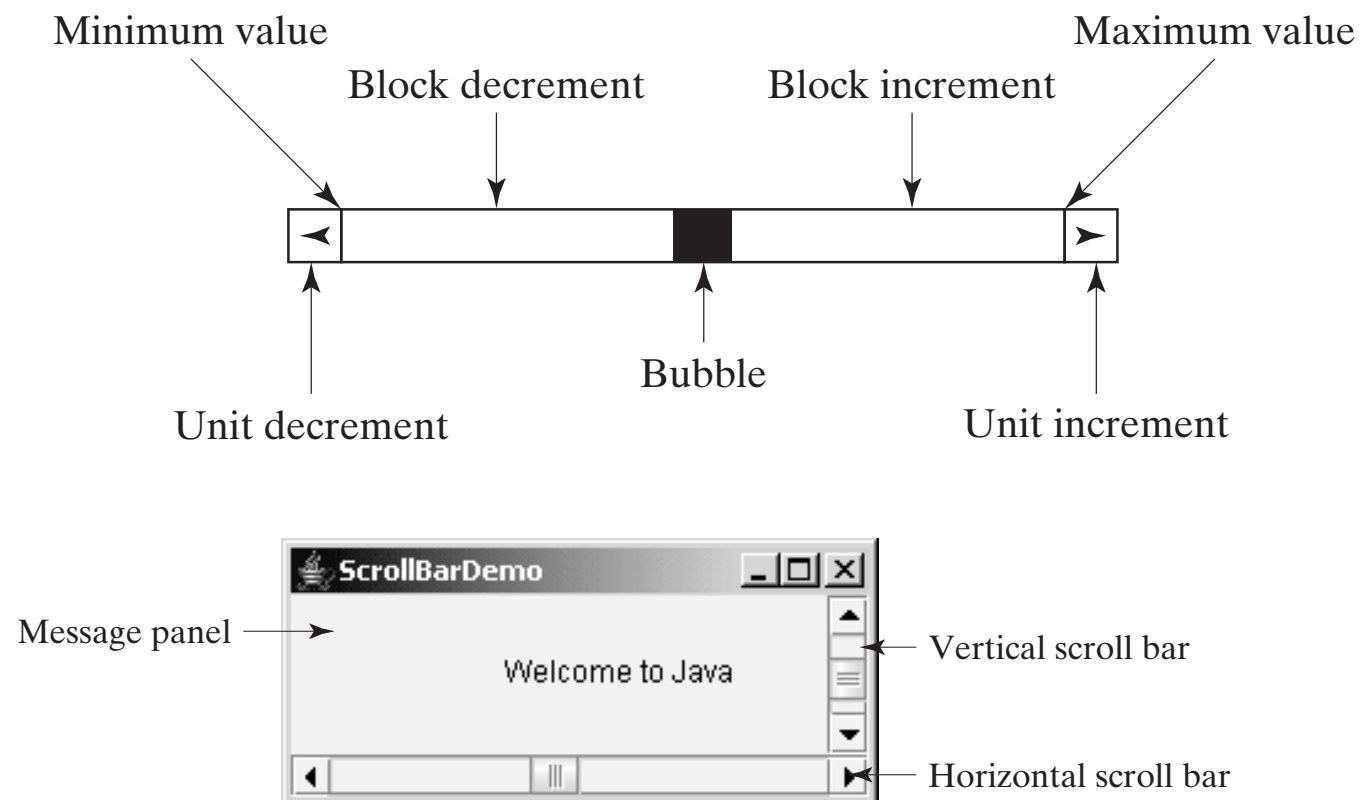
(b) Single-interval selection



(c) Multiple-interval selection

- ListDemo.java

ScrollBar



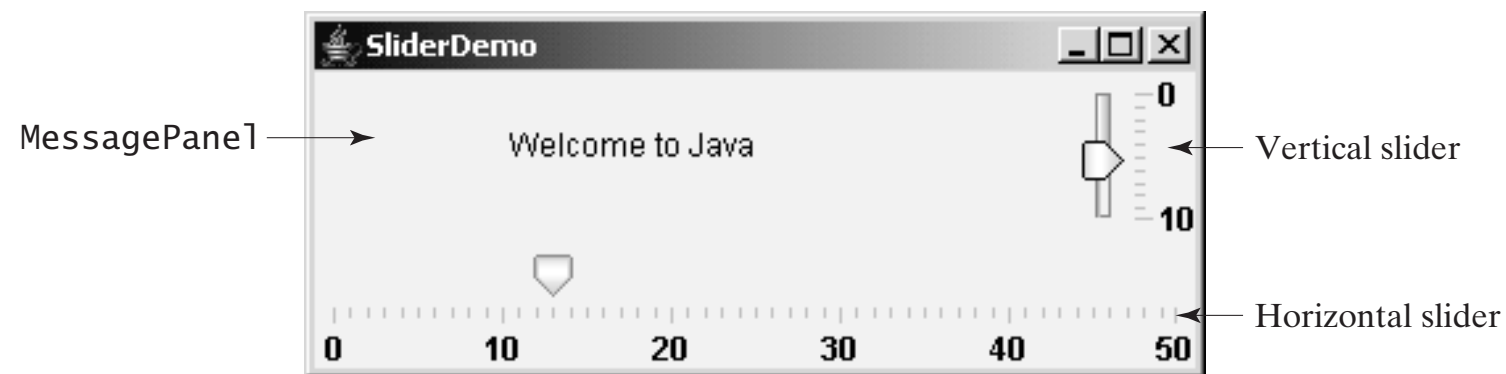
- **ScrollBarDemo.java**

javax.swing.JScrollBar

```
-orientation: int  
-maximum: int  
  
-minimum: int  
  
-visibleAmount: int  
  
-value: int  
-blockIncrement: int  
  
-unitIncrement: int
```

```
+JScrollBar()  
+JScrollBar(orientation: int)  
+JScrollBar(orientation: int, value:  
    int, extent: int, min: int, max: int)  
+addAdjustmentListener(listener:  
    AdjustmentListener): void
```

Slider



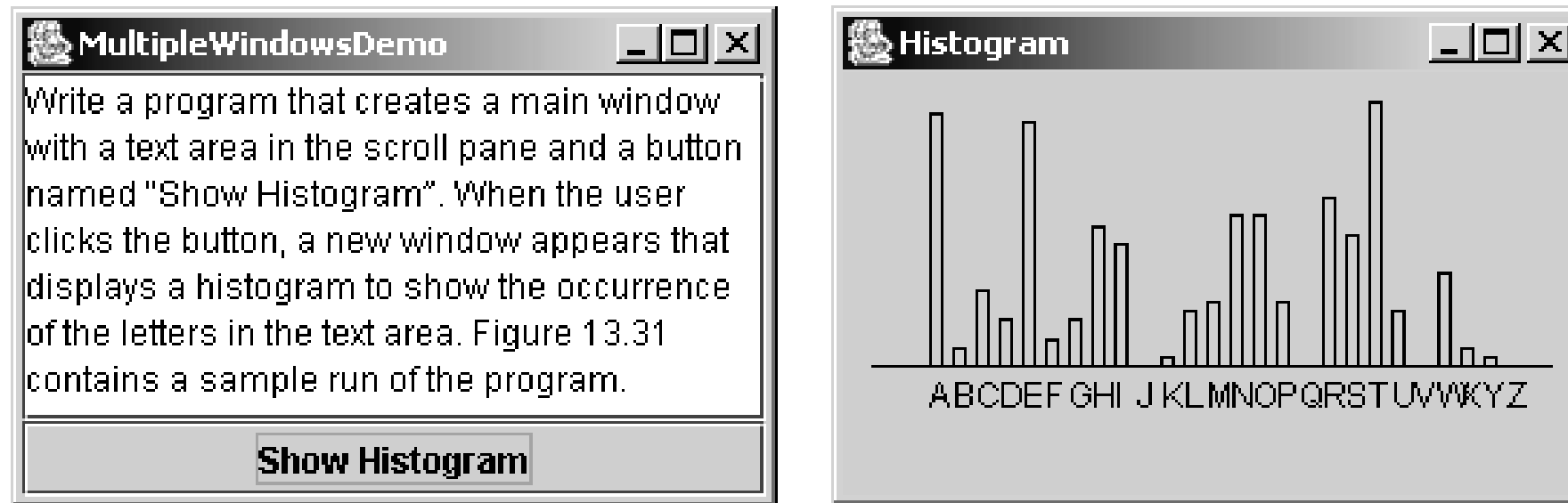
- **SliderDemo.java**

javax.swing.JSlider

-maximum: int
-minimum: int
-value: int
-orientation: int
-paintLabels: boolean
-paintTicks: boolean
-paintTrack: boolean
-majorTickSpacing: int
-minorTickSpacing: int
-inverted: boolean

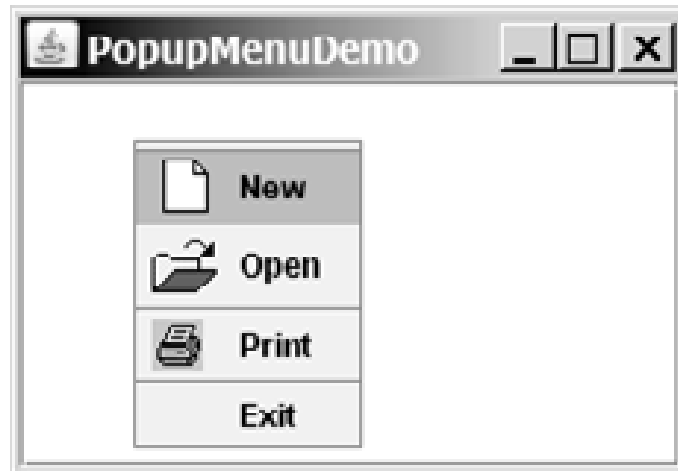
+JSlider()
+JSlider(min: int, max: int)
+JSlider(min: int, max: int, value: int)
+JSlider(orientation: int)
+JSlider(orientation: int, min: int, max: int, value: int)
+addChangeListener(listener: ChangeListener) :void

Multiple Windows



- `MultipleWindowsDemo.java`
- `Histogram.java`

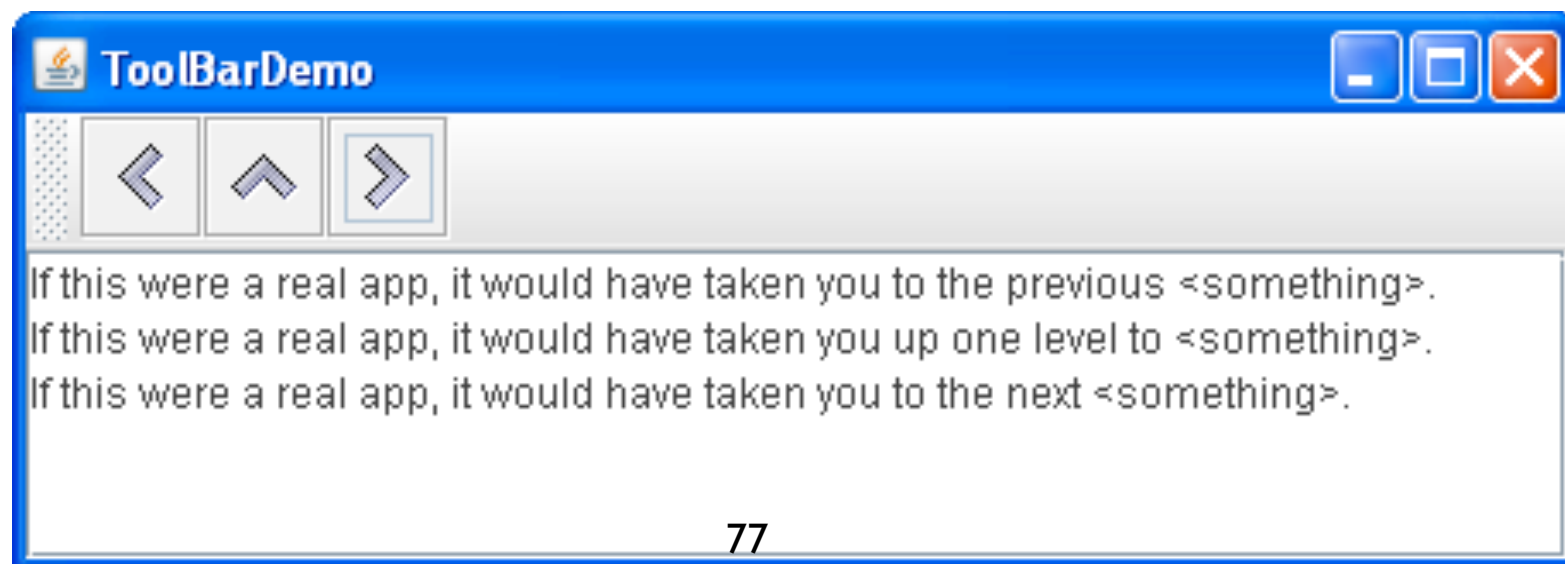
PopupMenu



- PopupMenuDemo.java

ToolBar

- A JToolBar is a container that groups several components — usually buttons with icons — into a row or column. Often, tool bars provide easy access to functionality that is also in menus. How to Use Actions describes how to provide the same functionality in menu items and tool bar buttons.



To use JToolBar

- Create a JToolBar, add it to the JFrame
- Create and add JButtons and other components to the JToolBar
- Add ActionListeners to the JButtons

ToolBarDemo.java

Dialog

Dialogs

- Modal dialogs block all other interaction.
 - Forces a response from the user.
- Non-modal dialogs allow other interaction.
 - This is sometimes desirable.
 - May be difficult to avoid inconsistencies.

JDialog

- Every dialog is dependent on a frame. When that frame is destroyed, so are its dependent dialogs. When the frame is iconified, its dependent dialogs disappear from the screen. When the frame is deiconified, its dependent dialogs return to the screen. The Swing automatically provides this behavior.
- A dialog can be modal. When a modal dialog is visible, it blocks user input to all other windows in the program. The JDialogs that JOptionPane creates are modal. To create a non-modal dialog, you must use the JDialog class directly.

Code for JDialog

- To define a dialog:

```
public class DlgReport extends JDialog {  
    public DlgReport(JFrame frame... ) {  
        super(frame,"The Title",true);  
    }  
}
```

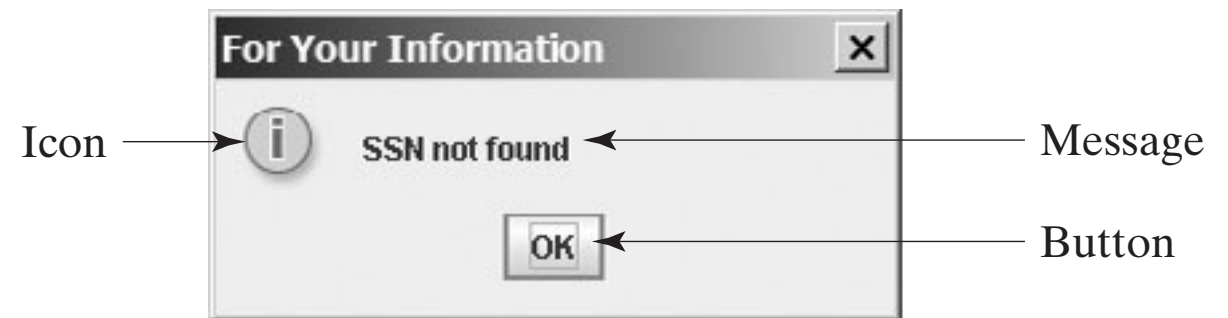
- To use that dialog:

```
DlgReport dlg = new DlgReport(theFrame,...);  
dlg.pack();  
dlg.setVisible(true);
```

Code for Dialogs

- Write your own code to describe the components in the dialog, and define the event handlers
- Write get/set functions to access the variables

JOptionPane standard dialogs



- Message dialog
 - Message text plus an OK button.
- Confirm dialog
 - Yes, No, Cancel options.
- Input dialog
 - Message text and an input field.
- Variations are possible.

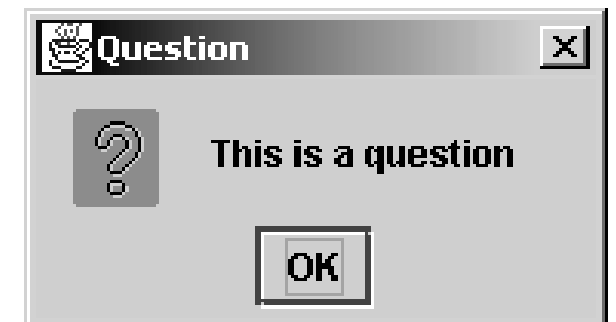
JOptionPane

- JOptionPane makes it easy to pop up a standard dialog box that prompts users for a value or informs them of something.

Method Name	Description
showConfirmDialog	Asks a confirming question, like yes/no/cancel.
showInputDialog	Prompt for some input.
showMessageDialog	Tell the user about something that has happened
showOptionDialog	The Grand Unification of the above three.

Message Dialogs

- JOptionPane.ERROR_MESSAGE
- JOptionPane.INFORMATION_MESSAGE
- JOptionPane.PLAIN_MESSAGE
- JOptionPane.WARNING_MESSAGE
- JOptionPane.QUESTION_MESSAGE



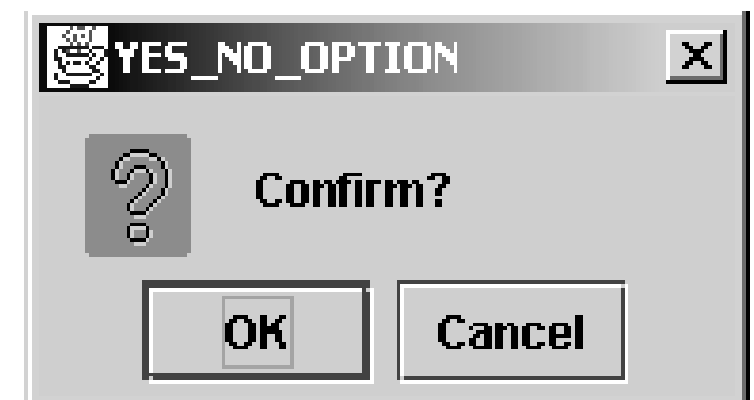
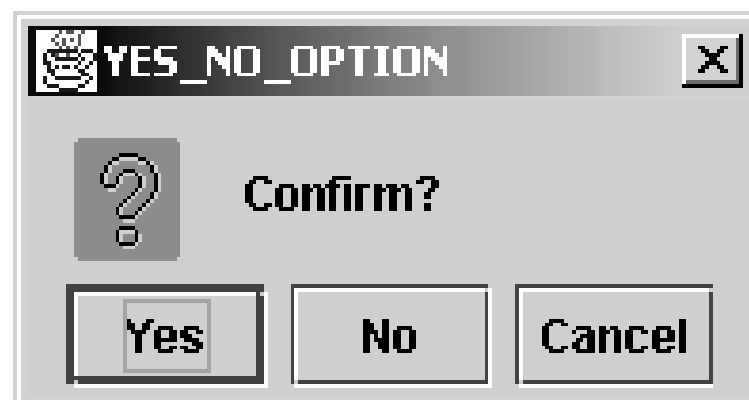
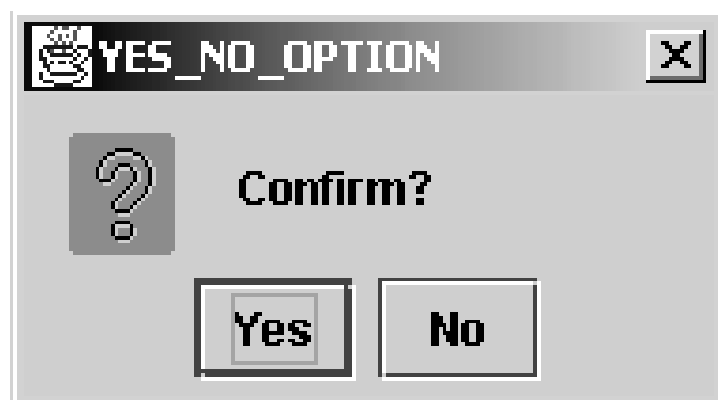
A message dialog

```
private void showAbout()  
{  
    JOptionPane.showMessageDialog(frame,  
        "ImageViewer\n" + VERSION,  
        "About ImageViewer",  
        JOptionPane.INFORMATION_MESSAGE);  
}
```

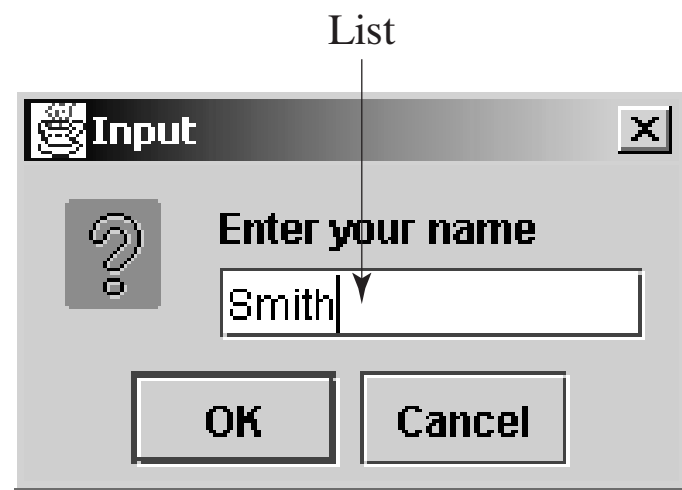


ConfirmDialogs

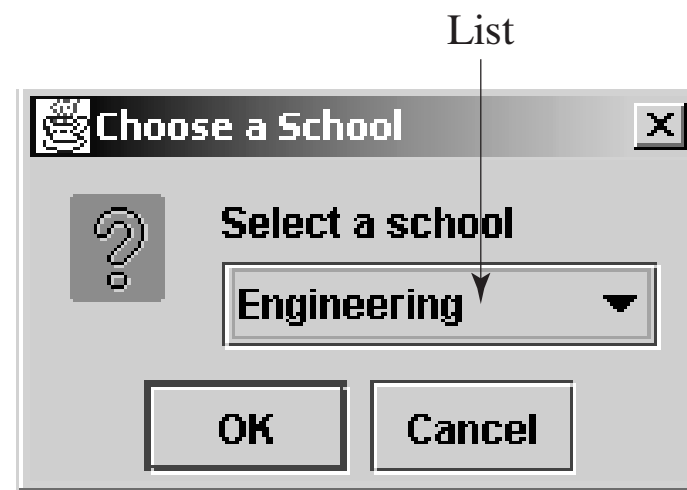
- JOptionPane.YES_NO_OPTION
- JOptionPane.YES_NO_CANCEL_OPTION
- JOptionPane.OK_CANCEL_OPTION



InputDialogs



(a) Text field



(b) Combo box



(c) List

Code for JOptionPane

```
JOptionPane.showMessageDialog(theFrame,"(C)2007 BA5AG");
```

```
String name = JOptionPane.showInputDialog(theFrame,"Input the name:","Search  
Author",JOptionPane.QUESTION_MESSAGE);
```

```
if ( JOptionPane.showConfirmDialog( this, "delete?", "delete?",  
JOptionPane.YES_NO_OPTION ) == JOptionPane.NO_OPTION )
```

JOptionPaneDemo.java

Create Your Dialog

- extend class JDialog
- set modal
- set visible
- dispose

```
public ColorDialog(java.awt.Frame parent, boolean modal) {  
    super(parent, modal);  
}
```

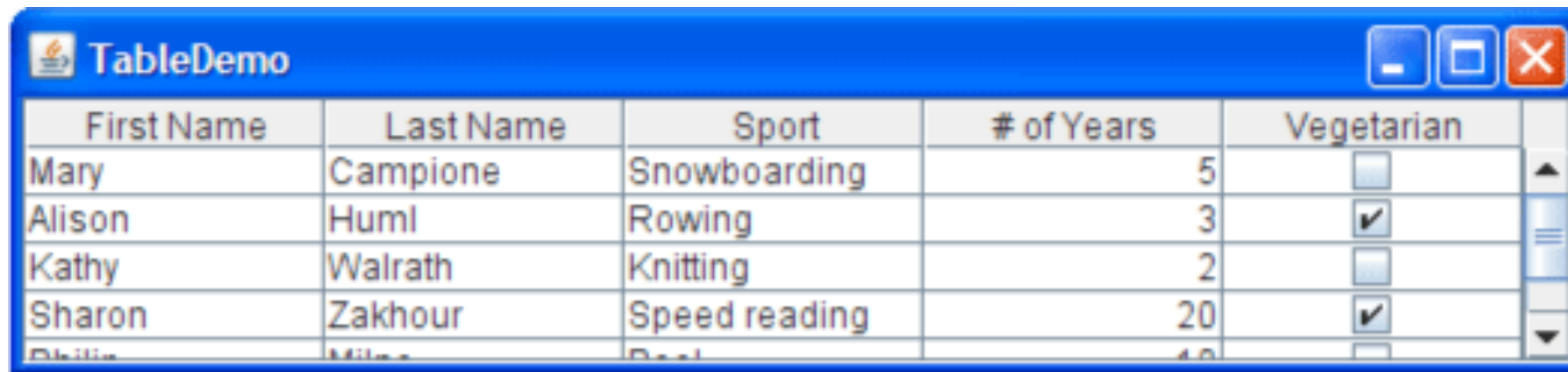
ColorDialog.java

JTable

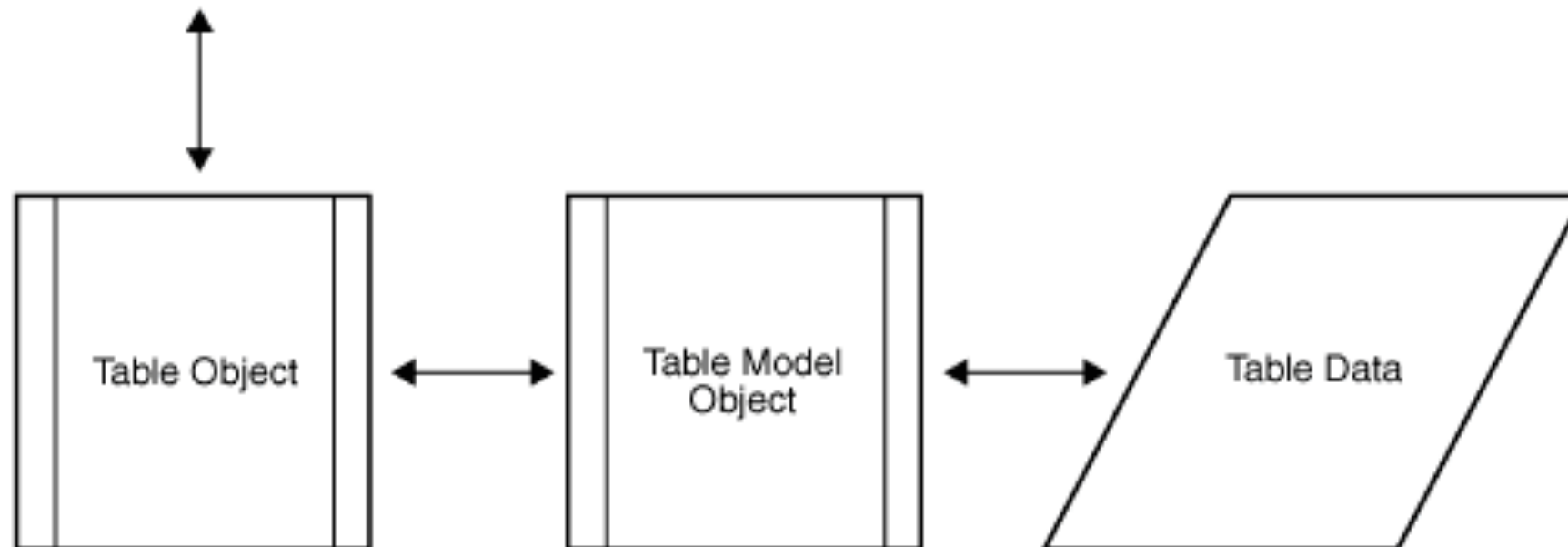
- With the JTable class you can display tables of data, optionally allowing the user to edit the data. JTable does not contain or cache data; it is simply a view of your data.



TableModel



First Name	Last Name	Sport	# of Years	Vegetarian
Mary	Campione	Snowboarding	5	☐
Alison	Huml	Rowing	3	☒
Kathy	Walrath	Knitting	2	☐
Sharon	Zakhour	Speed reading	20	☒
Phillip	Miles	Reading	10	☐



Code for JTable

```
JTable table = new JTable(new MyDataModel());  
JScrollPane scrollPane = new JScrollPane(table);  
add(scrollPane, BorderLayout.CENTER);
```

AbstractTableModel

public int getColumnCount()

public int getRowCount()

public String getColumnName(int col)

public Object getValueAt(int row, int col)

public boolean isCellEditable(int row, int col)

public void setValueAt(Object value, int row, int col)

The imageviewer project

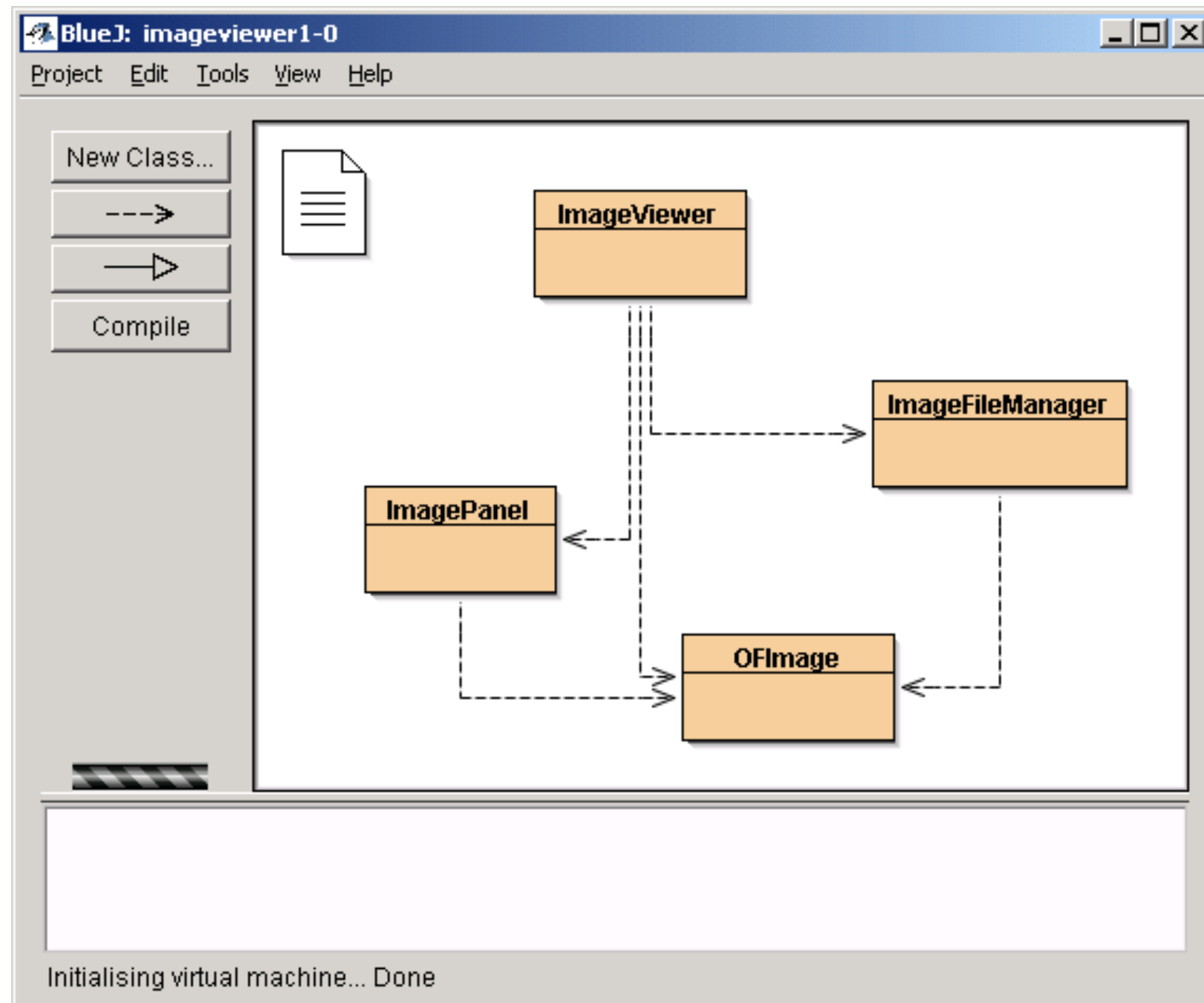
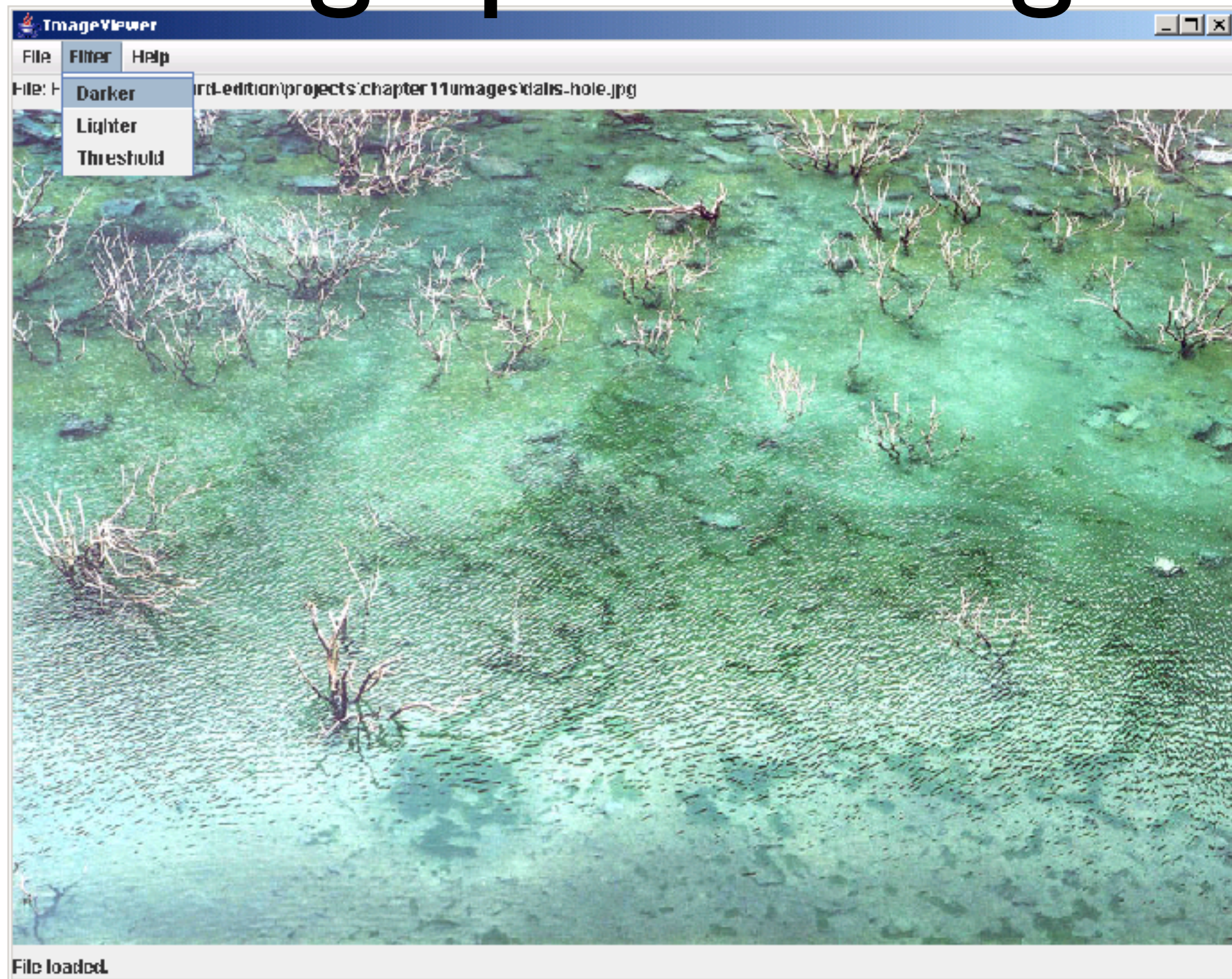


Image processing



Class responsibilities

- `ImageViewer`
 - Sets up the GUI structure.
- `ImageFileManager`
 - Static methods for image file loading and saving.
- `ImagePanel`
 - Displays the image within the GUI.
- `OFImage`
 - Models a 2D image.

OFlmage

- Our subclass of `BufferedImage`.
- Represents a 2D array of pixels.
- Important methods:
 - `getPixel`, `setPixel`
 - `getWidth`, `getHeight`
- Each pixel has a color.
 - We use `java.awt.Color`.

Adding an ImagePanel

```
public class ImageViewer
{
    private JFrame frame;
    private ImagePanel imagePanel;

    ...

    private void makeFrame()
    {
        Container contentPane = frame.getContentPane();
        imagePanel = new ImagePanel();
        contentPane.add(imagePanel);
    }

    ...
}
```

Loading an image

```
public class ImageViewer
{
    private JFrame frame;
    private ImagePanel imagePanel;

    ...

    private void openFile()
    {
        File selectedFile = ...;
        OFImage image =
            ImageFileManager.loadImage(selectedFile);
        imagePanel.setImage(image);
        frame.pack();
    }

    ...
}
```

Image filters

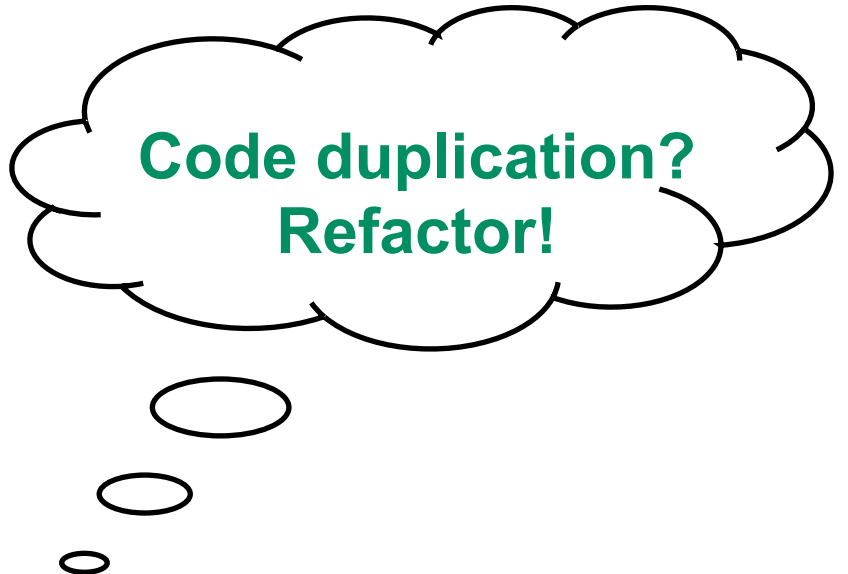
- Functions applied to the whole image.

```
int height = getHeight();
int width = getWidth();
for(int y = 0; y < height; y++) {
    for(int x = 0; x < width; x++) {
        Color pixel = getPixel(x, y);
        alter the pixel's color value;
        setPixel(x, y, pixel);
    }
}
```

Adding further filters

```
private void makeLighter()
{
    if(currentImage != null) {
        currentImage.lighter();
        frame.repaint();
        showStatus("Applied: lighter");
    }
    else {
        showStatus("No image loaded.");
    }
}
```

```
private void threshold()
{
    if(currentImage != null) {
        currentImage.threshold();
        frame.repaint();
        showStatus("Applied: threshold");
    }
    else {
        showStatus("No image loaded.");
    }
}
```

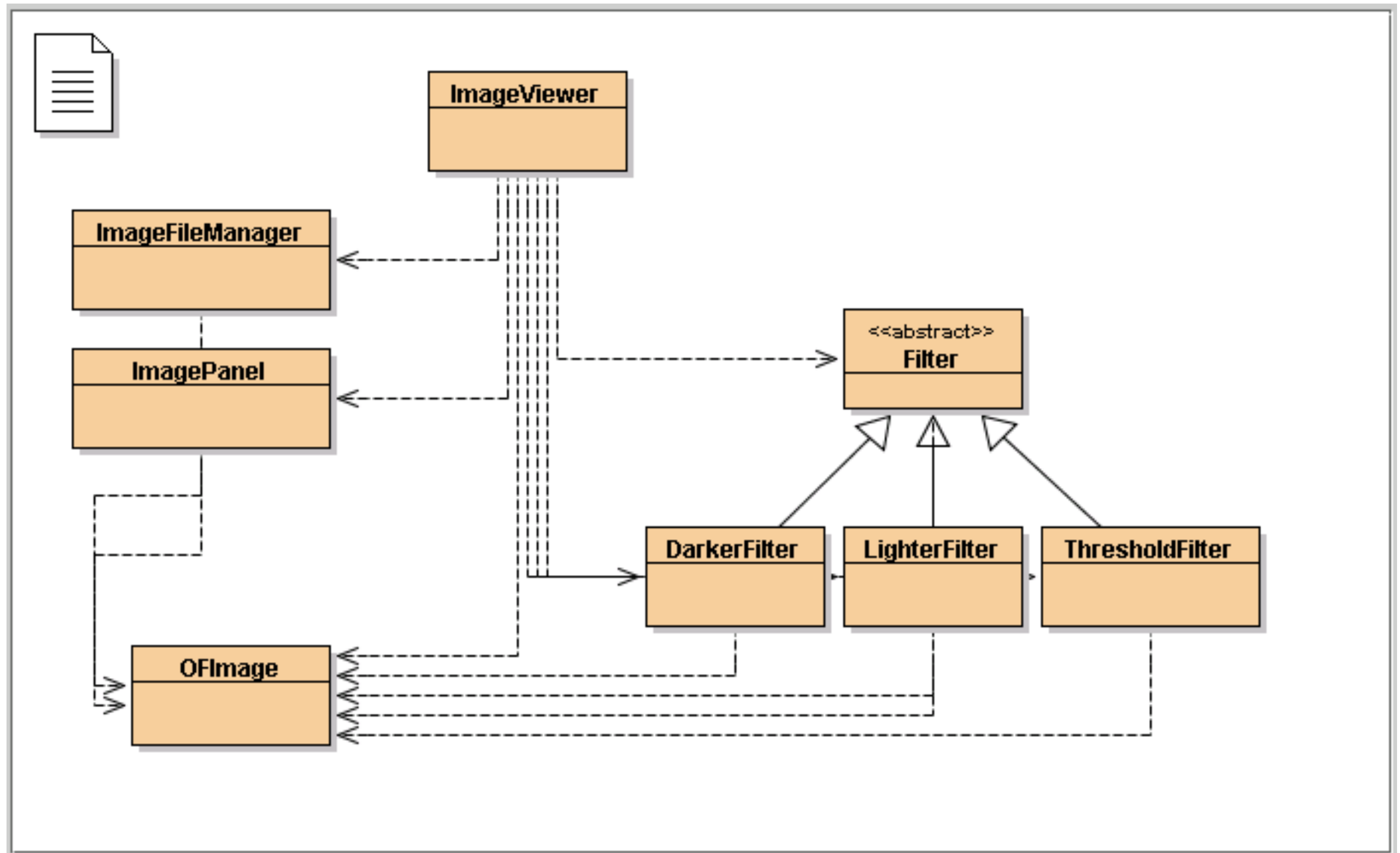


**Code duplication?
Refactor!**

Adding further filters

- Define a `Filter` superclass (abstract).
- Create function-specific subclasses.
- Create a collection of subclass instances in `ImageViewer`.
- Define a generic `applyFilter` method.
- See *imageviewer2-0*.

imageviewer2-0



Buttons and nested layouts

A GridLayout inside
a FlowLayout inside
a BorderLayout.



Borders

- Used to add decoration around components.
- Defined in `javax.swing.border`
 - `BevelBorder`, `CompoundBorder`,
`EmptyBorder`, `EtchedBorder`,
`TitledBorder`.

Adding spacing

```
JPanel contentPane = (JPanel)frame.getContentPane();  
contentPane.setBorder(new EmptyBorder(6, 6, 6, 6));
```

```
// Specify the layout manager with nice spacing  
contentPane.setLayout(new BorderLayout(6, 6));
```

```
imagePanel = new ImagePanel();  
imagePanel.setBorder(new EtchedBorder());  
contentPane.add(imagePanel, BorderLayout.CENTER);
```